

Deep Reinforcement Learning and Control

Imitation Learning I: Behavior Cloning and GAIL

Spring 2025, CMU 10-403

Instructors: Katerina Fragkiadaki and Aviral Kumar

Limitations of Learning by Interaction

- The agent should have the chance to try (and fail) MANY times
- This is hard when safety is a concern: we cannot afford to fail
- This is also quite hard in general in real life where each interaction takes time (in contrast to simulation)



Crusher robot

Learning from Demonstration for Autonomous Navigation in Complex Unstructured Terrain, Silver et al. 2010

Imitation Learning (a.k.a. Learning from Demonstrations)

visual imitation



The actions of the teacher need to be inferred from visual sensory input and mapped to the action space of the agent.

Two challenges:

- 1) visual understanding
- 2) action mapping, especially when the agent and the teacher do not have the same action space

(later lecture)

kinesthetic imitation



- The teacher takes over the end-effectors of the agent.
- Demonstrated actions are in the action space of the imitator and can be imitated directly)

this lecture

Challenges in Imitation Learning

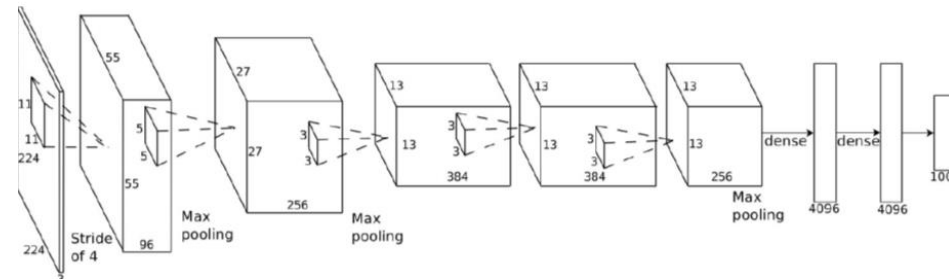
- State distribution shift between train and test sets
- Multi-modality of state-conditioned action distributions
- Generalization (as for all supervised learning problems)

Challenges in Imitation Learning

- State distribution shift between train and test sets
- Multi-modality of state-conditioned action distributions
- Generalization (as for all supervised learning problems)

Video sequence labelling

Action labelling: a mapping from states/observations to action labels



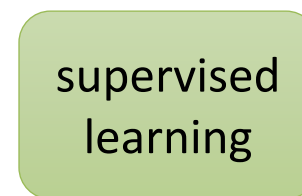
“picking up cereal A”

$$f_{\theta}(y_t | \mathbf{o}_t)$$

- Assume action labels in an annotated video are i.i.d. (independent and identically distributed).
- Train a classifier to map observations to labels at each time step of the trajectory



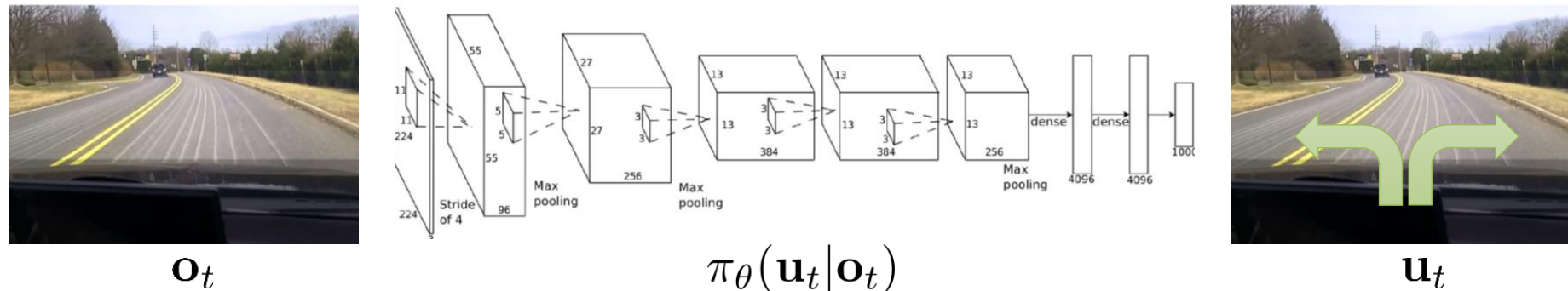
$\mathbf{o}_t$
 y_t



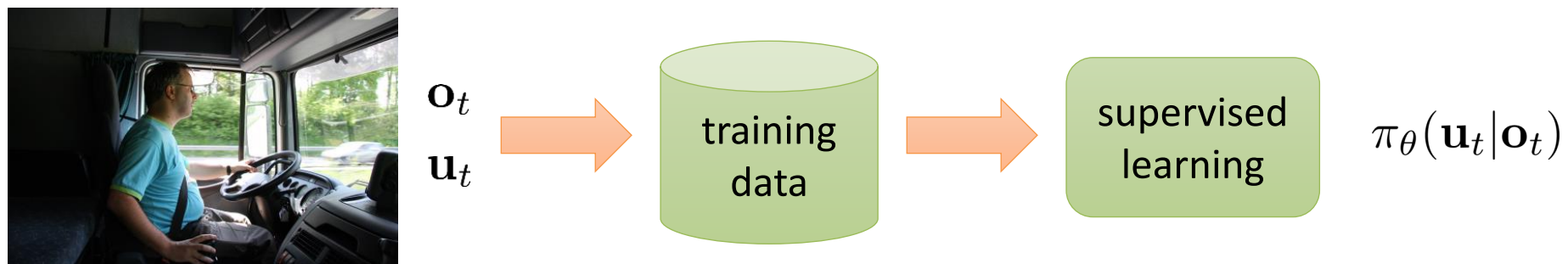
$$f_{\theta}(y_t | \mathbf{o}_t)$$

Imitation Learning

Policy: a mapping from observations to actions

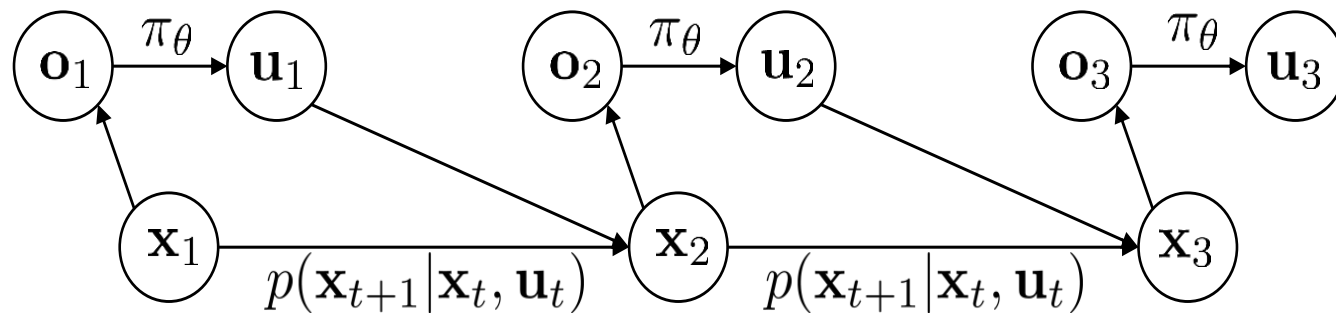


- Assume actions in the expert trajectories are i.i.d. (independent and identically distributed)
- Train a function to map observations/states to actions at each time step of the trajectory



Imitation learning VS Sequence labelling

Imitation learning



$\mathbf{u}_t$:the action at time t

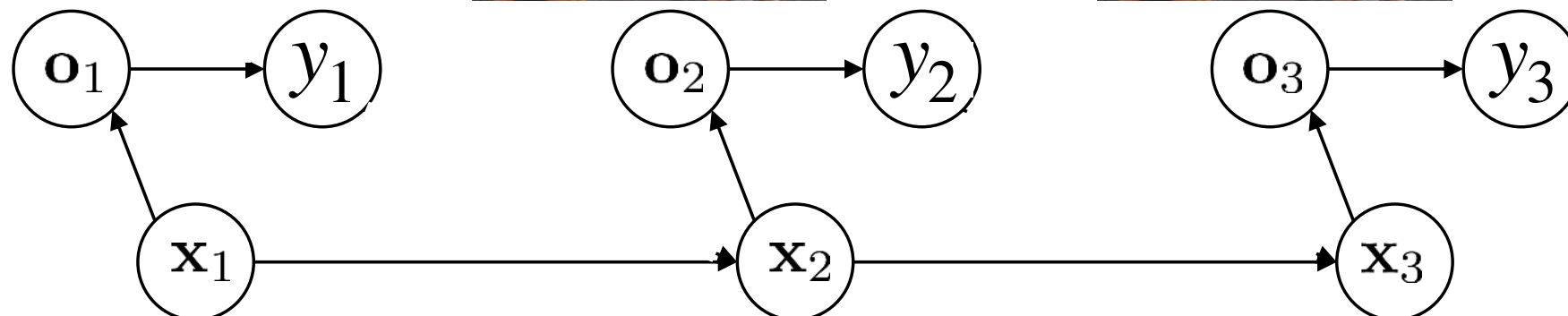
$\mathbf{o}_t$:the observation at time t

$\mathbf{x}_t$:the state at time t

Training data:

$o_1^1, u_1^1, o_2^1, u_2^1, o_3^1, u_3^1, \dots$
 $o_1^2, u_1^2, o_2^2, u_2^2, o_3^2, u_3^2, \dots$
 $o_1^3, u_1^3, o_2^3, u_2^3, o_3^3, u_3^3, \dots$

Sequence labelling



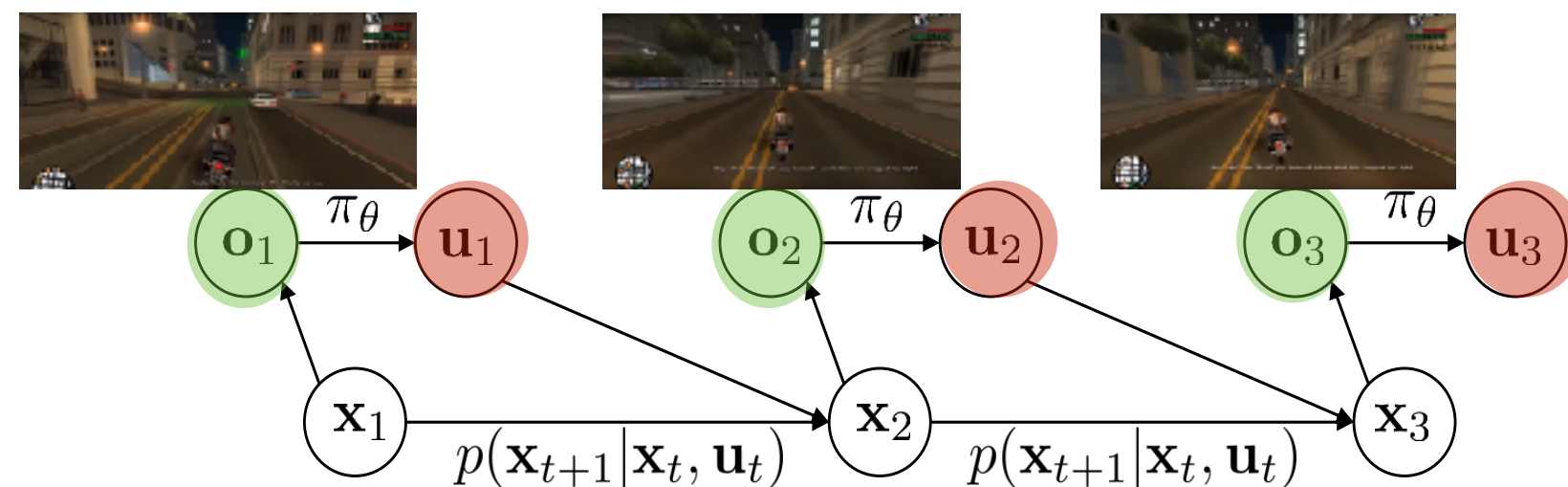
y_t : which product was purchased at frame t (if any) $\mathbf{o}_t$:the observation at time t $\mathbf{x}_t$:the state at time t

Training data:

$o_1^1, y_1^1, o_2^1, y_2^1, o_3^1, y_3^1, \dots$
 $o_1^2, y_1^2, o_2^2, y_2^2, o_3^2, y_3^2, \dots$
 $o_1^3, y_1^3, o_2^3, y_2^3, o_3^3, y_3^3, \dots$

Imitation learning VS Sequence labelling

Imitation learning



Training data:

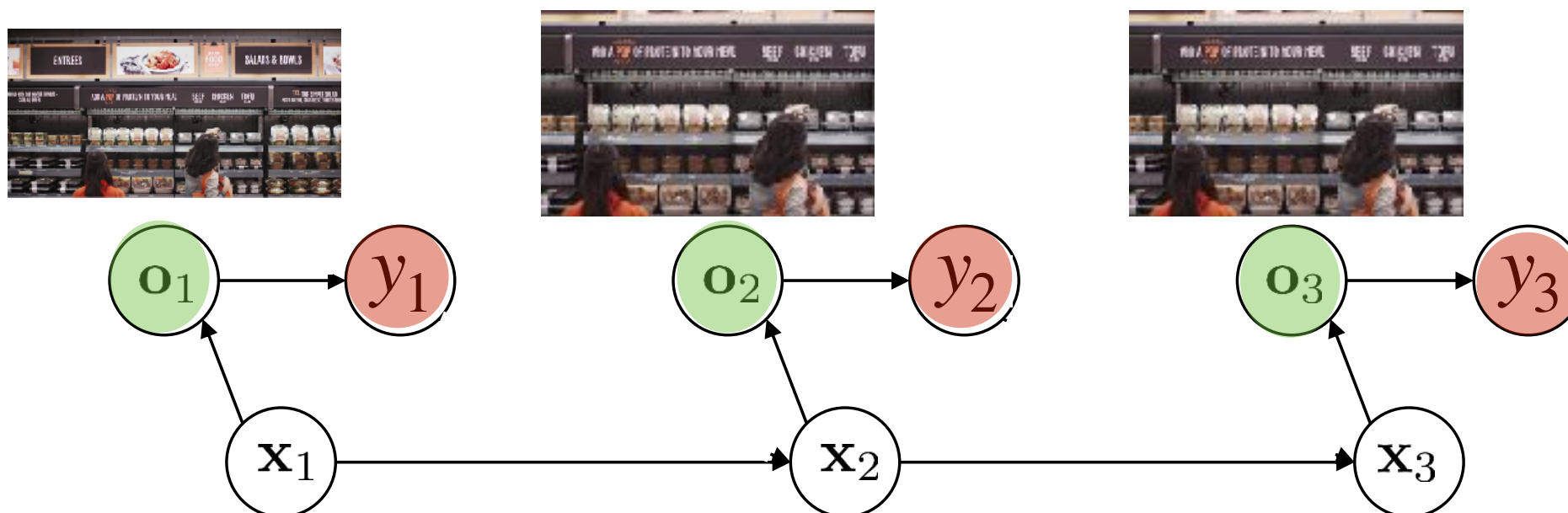
$o_1^1, u_1^1, o_2^1, u_2^1, o_3^1, u_3^1, \dots$
 $o_1^2, u_1^2, o_2^2, u_2^2, o_3^2, u_3^2, \dots$
 $o_1^3, u_1^3, o_2^3, u_2^3, o_3^3, u_3^3, \dots$

u_t : the action at time t

o_t : the observation at time t

x_t : the state at time t

Sequence labelling



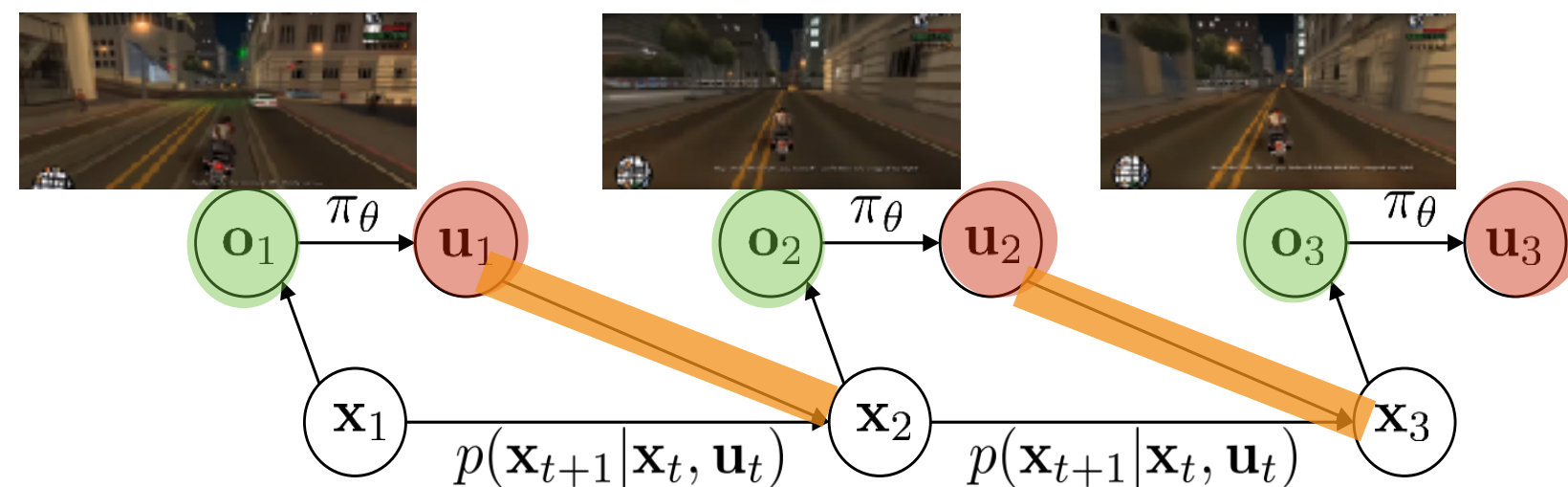
Training data:

$o_1^1, y_1^1, o_2^1, y_2^1, o_3^1, y_3^1, \dots$
 $o_1^2, y_1^2, o_2^2, y_2^2, o_3^2, y_3^2, \dots$
 $o_1^3, y_1^3, o_2^3, y_2^3, o_3^3, y_3^3, \dots$

y_t : which product was purchased at frame t (if any) o_t : the observation at time t x_t : the state at time t

Imitation learning VS Sequence labelling

Imitation learning



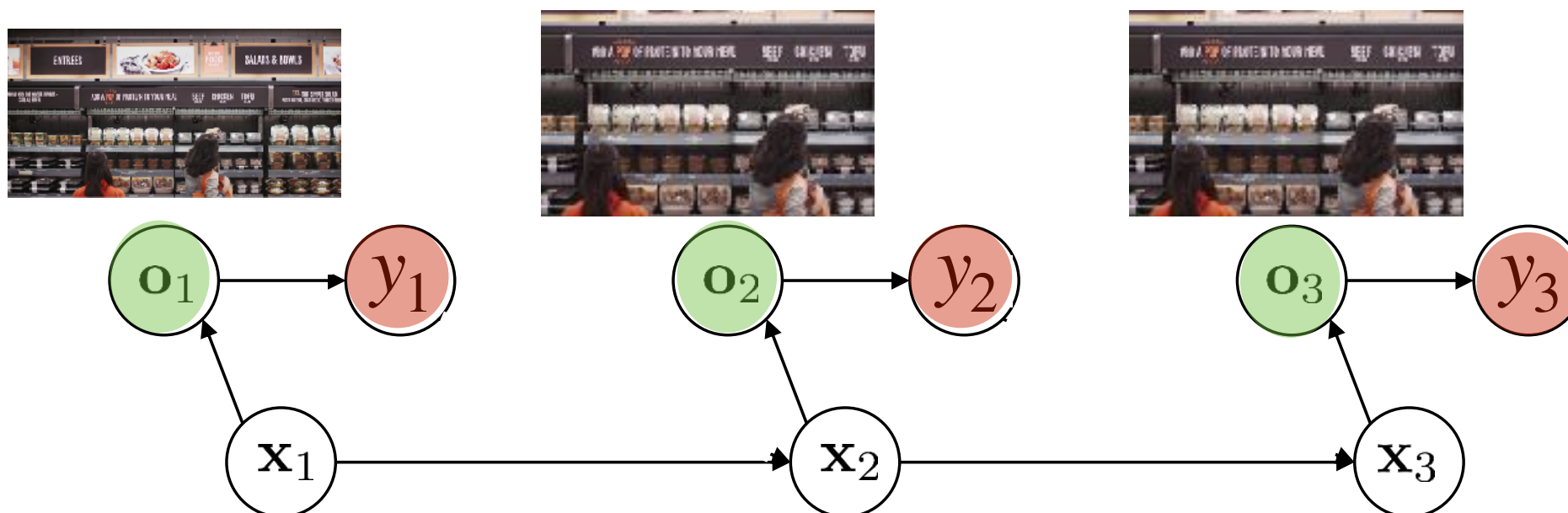
- In RL, our actions will influence our future state, and thus our future data.
- In sequence labelling, our labels won't influence the future frames.

u_t : the action at time t

o_t : the observation at time t

x_t : the state at time t

Sequence labelling



Training data:

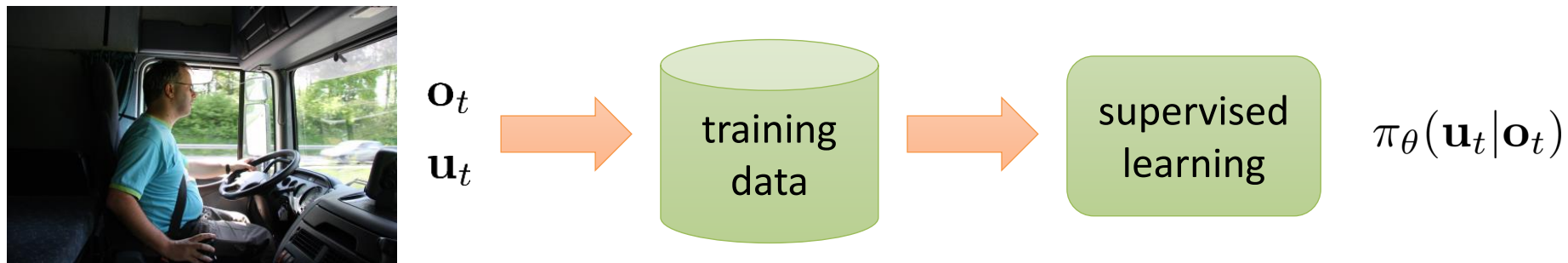
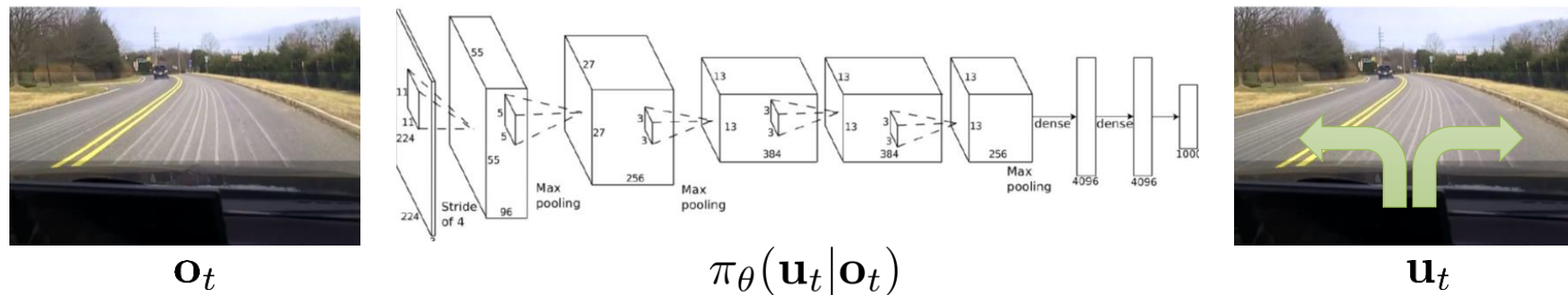
$o_1^1, y_1^1, o_2^1, y_2^1, o_3^1, y_3^1, \dots$

$o_1^2, y_1^2, o_2^2, y_2^2, o_3^2, y_3^2, \dots$

$o_1^3, y_1^3, o_2^3, y_2^3, o_3^3, y_3^3, \dots$

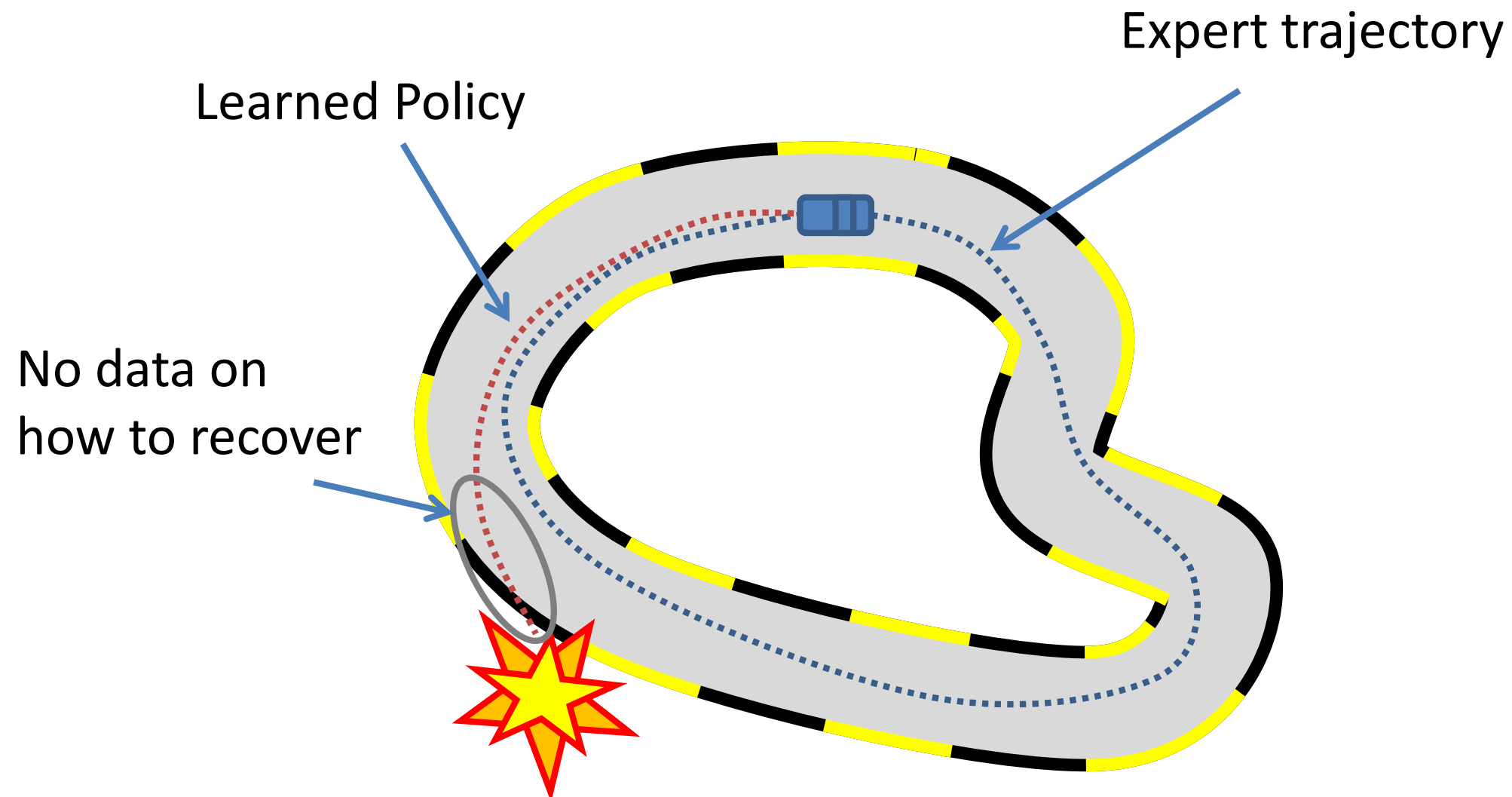
y_t : which product was purchased at frame t (if any)

Challenge: Distribution Shift in Imitation Learning



Distribution mismatch (distribution shift)

$$P_{\pi^*}(\mathbf{o}_t) \neq P_{\pi_\theta}(\mathbf{o}_t)$$



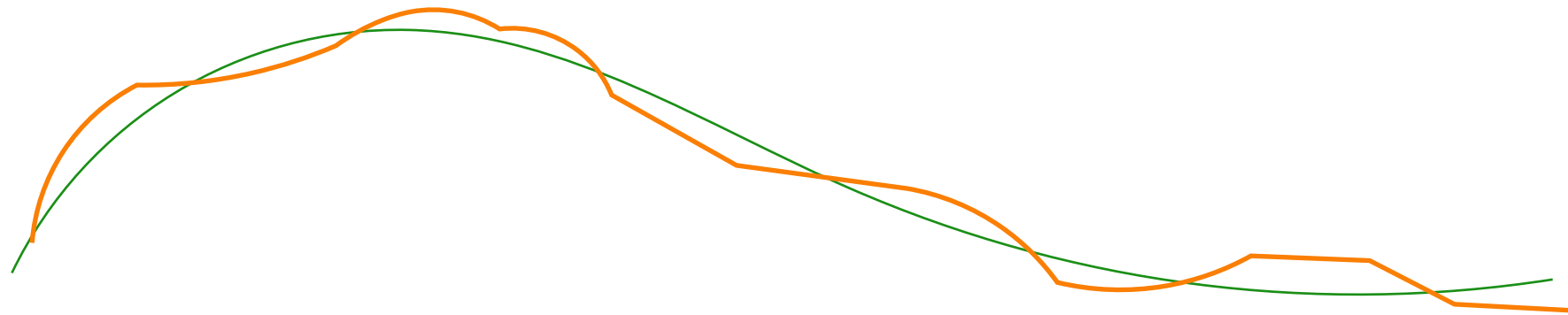
Distribution mismatch (distribution shift)

	supervised learning	supervised learning + control (NAIVE)
train	$(x,y) \sim D$	$\mathbf{o}_t \sim P_{\pi^*}(\mathbf{o}_t)$
test	$(x,y) \sim D$	$\mathbf{o}_t \sim P_{\pi_\theta}(\mathbf{o}_t)$

Supervised learning succeeds when training and test data distributions match, that is a fundamental assumption.

Independent in time errors

This means that at each time step t , the agent wakes up on a **state drawn from the state distribution of the expert trajectories**, and executes an action.

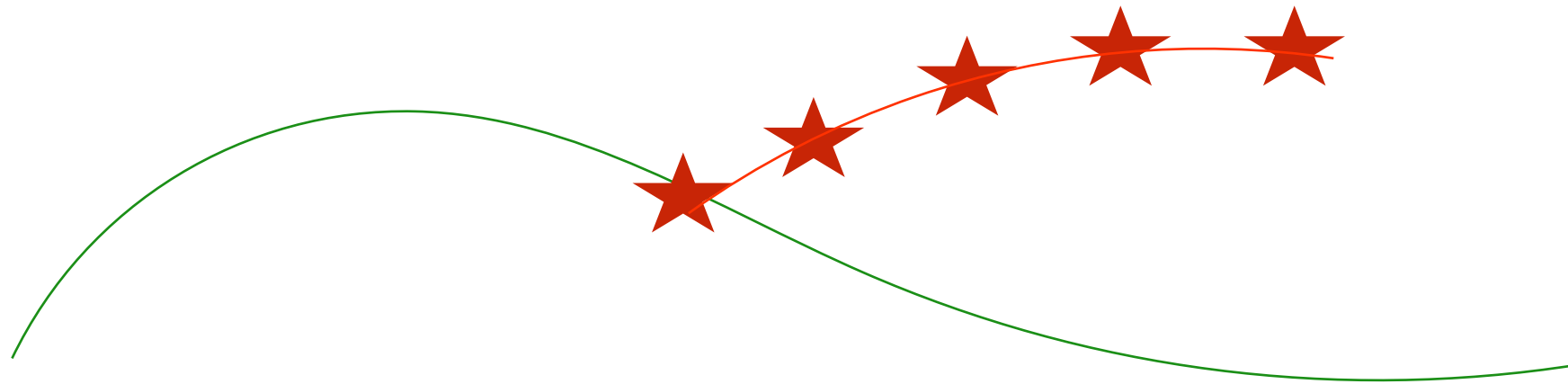


error at time t with probability ε

$E[\text{Total errors}] \approx \varepsilon T$, T the length of the trajectory

Compounding Errors

This means that at each time step t , the agent wakes up on a **state drawn from the state distribution resulting from executing the action the learned policy suggested in the previous time step.**



error at time t with probability ε

$$E[\text{Total errors}] \approx \varepsilon(T + (T-1) + (T-2) + \dots + 1) \propto \varepsilon T^2$$

Solution: data augmentations

Change $P_{\pi^*}(\mathbf{o}_t)$ by **augmenting** the expert demonstration trajectories.

Add examples in expert demonstration trajectories to cover the states/ observations points where the agent will land when trying out its own policy. How?

1. By generating synthetic data in simulation
2. By collecting additional data via clever hardware
3. By interactively querying the experts in additional datapoints

Solution: data augmentations

Change $P_{\pi^*}(\mathbf{o}_t)$ by **augmenting** the expert demonstration trajectories.

Add examples in expert demonstration trajectories to cover the states/ observations points where the agent will land when trying out its own policy. How?

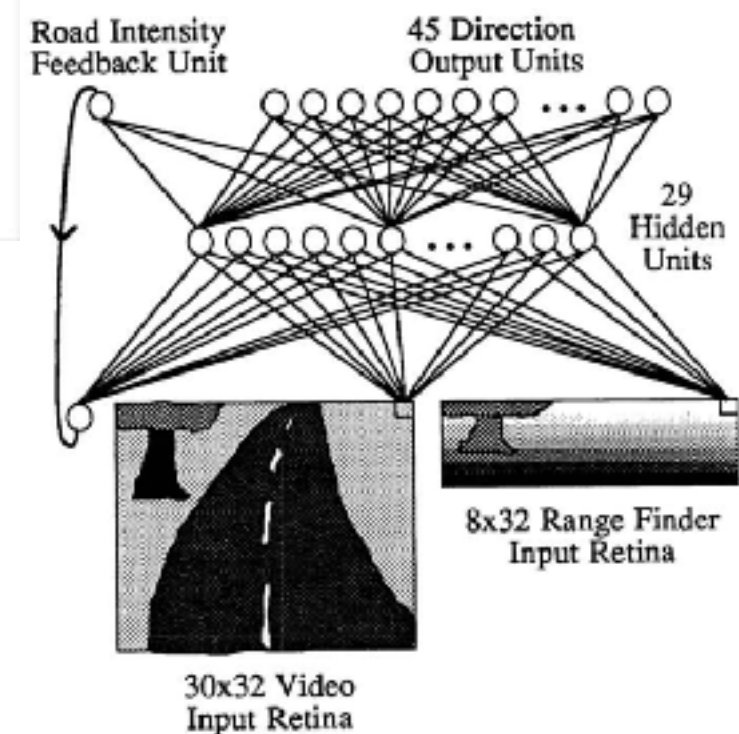
1. By generating synthetic data in simulation
2. By collecting additional data via clever hardware
3. By interactively querying the experts in additional datapoints

Demonstration Augmentation: ALVINN 1989

Neural Network-Based
Autonomous Driving

23 November 1992

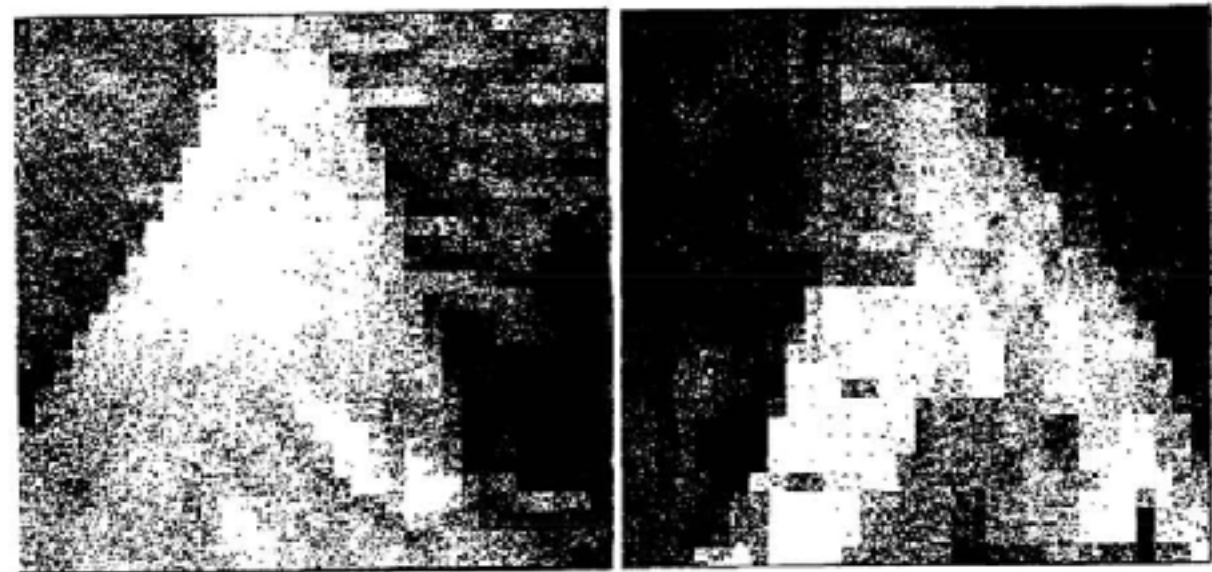
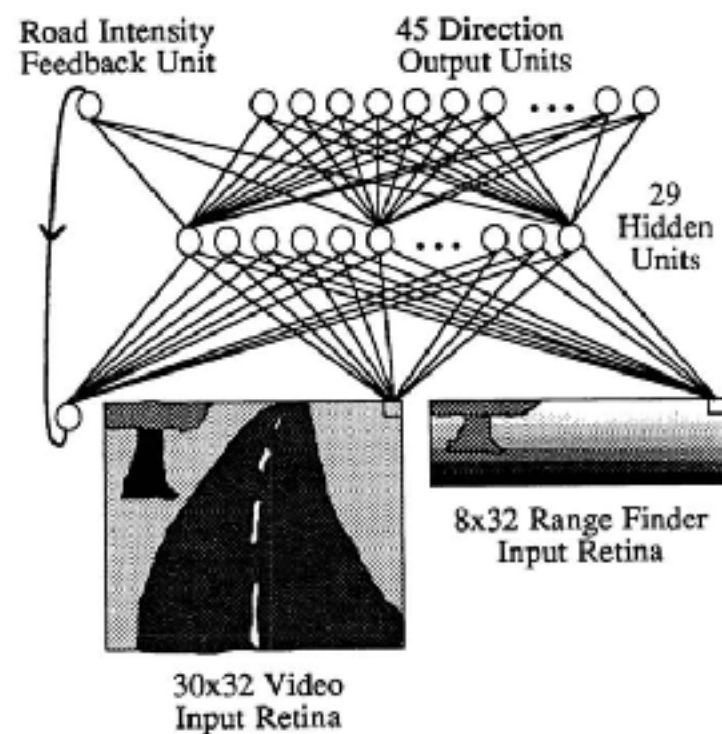
[Courtesy of Dean Pomerleau]



Demonstration Augmentation: ALVINN 1989

“In addition, the network must not solely be shown examples of accurate driving, but also how to recover (i.e. return to the road center) once a mistake has been made. Partial initial training on a variety of simulated road images should help eliminate these difficulties and facilitate better performance.”
ALVINN: An autonomous Land vehicle in a neural Network”,
Pomerleau 1989

Demonstration Augmentation: ALVINN 1989



Real Road Image

Simulated Road Image

- Using graphics simulator for road images and corresponding steering angle ground-truth
- Online adaptation to human driver steering angle control
- 3 layers, fully connected layers, very low resolution input from camera and range sensor

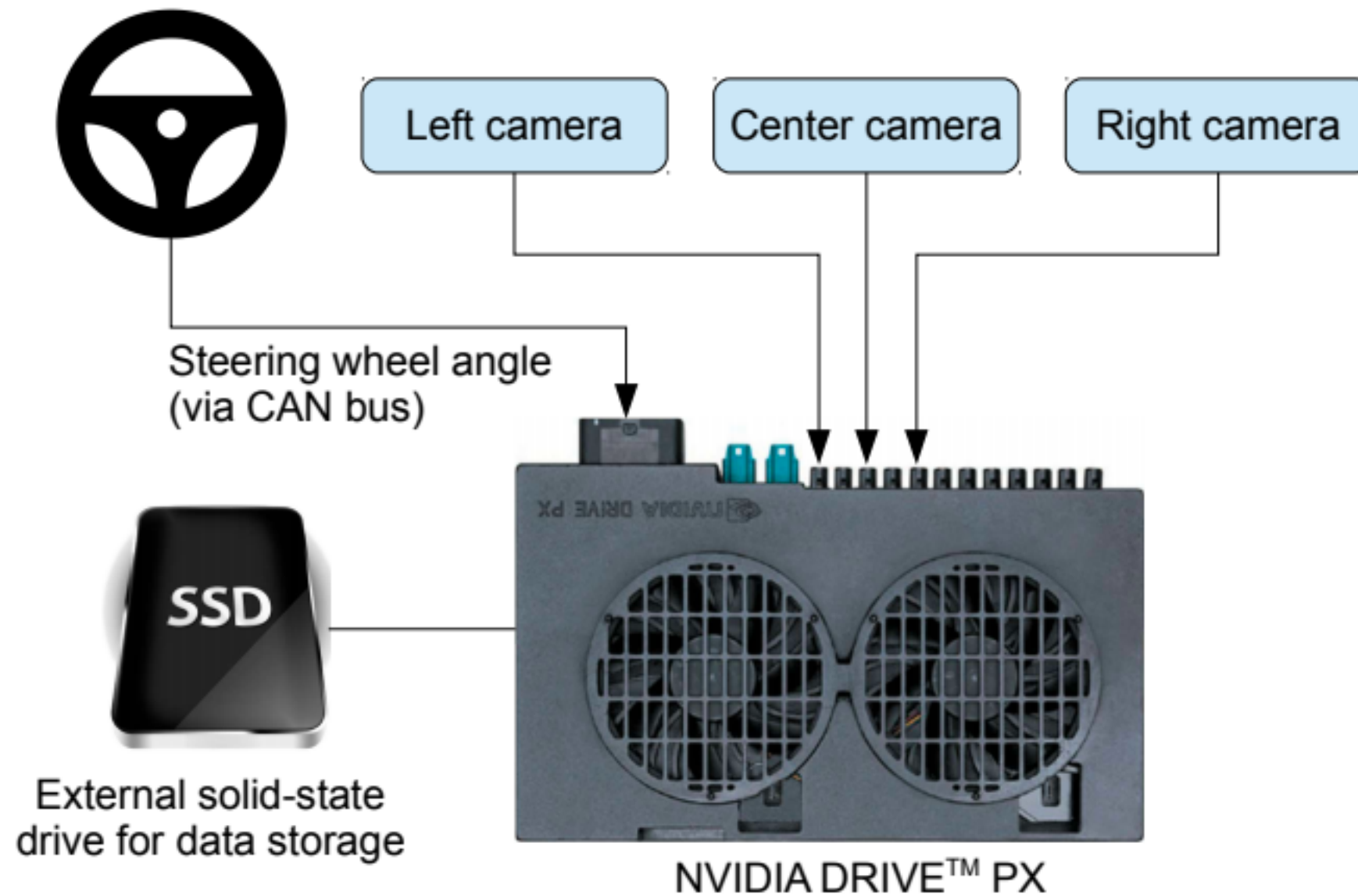
Solution: data augmentations

Change $P_{\pi^*}(\mathbf{o}_t)$ by **augmenting** the expert demonstration trajectories.

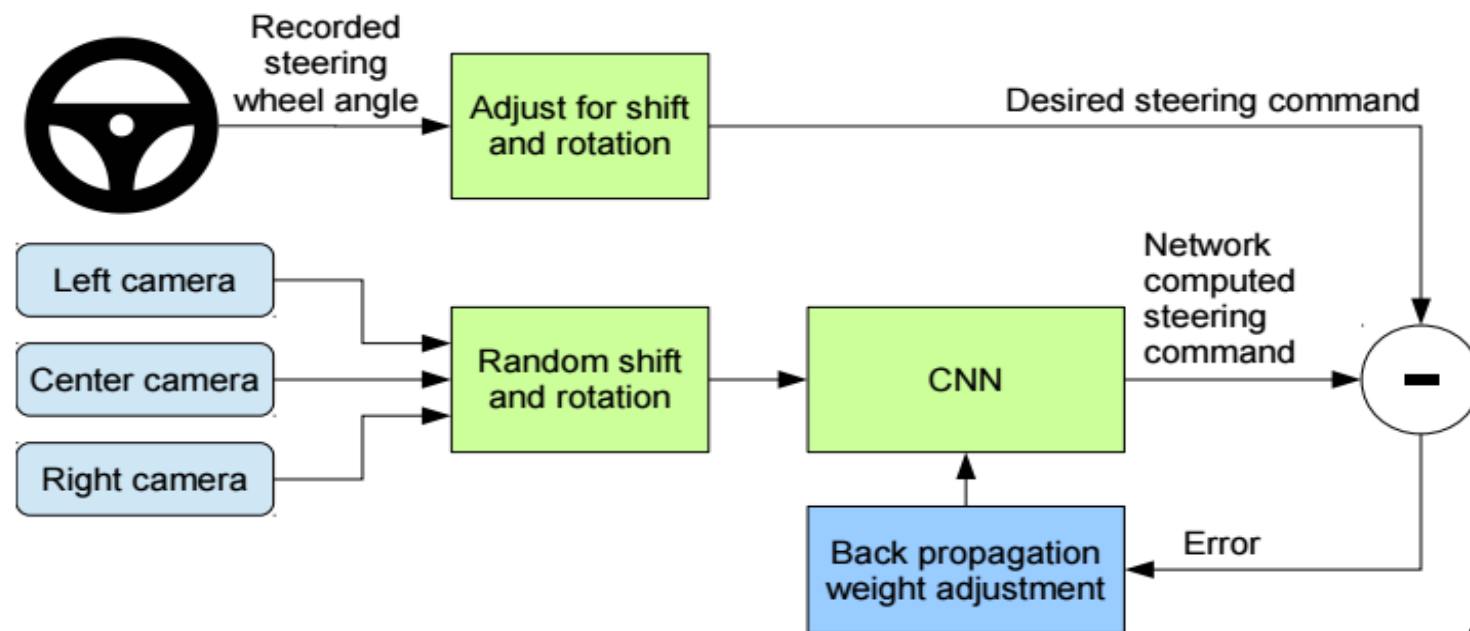
Add examples in expert demonstration trajectories to cover the states/observations points where the agent will land when trying out its own policy. How?

1. By generating synthetic data in simulation
2. By collecting additional data via clever hardware
3. By interactively querying the experts in additional datapoints

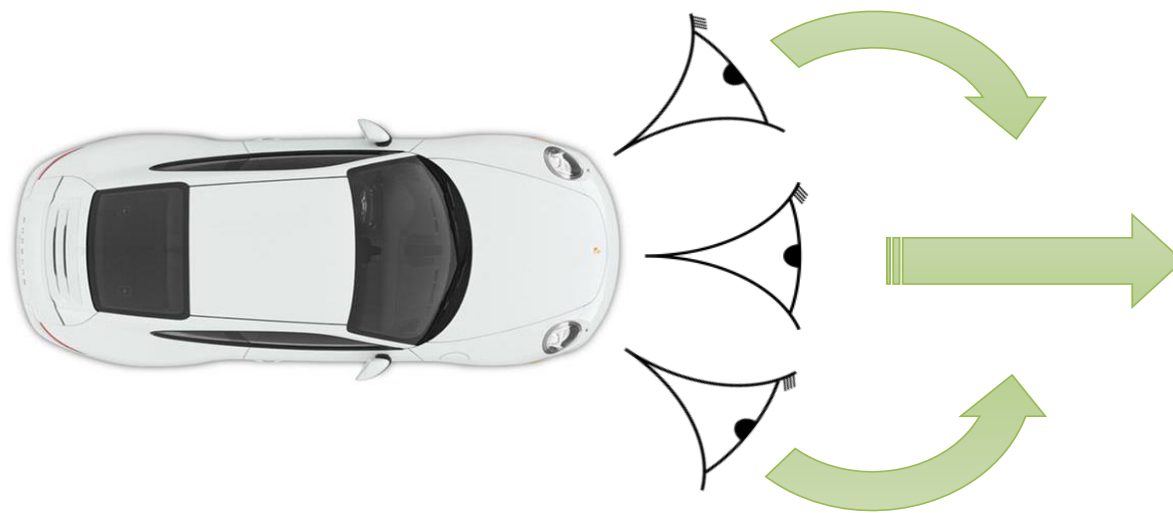
Learning to Drive a Car: Supervised Learning



Demonstration Augmentation: NVIDIA 2016



Additional, left and right cameras with automatic ground-truth labels to recover from mistakes



“DAVE-2 was inspired by the pioneering work of Pomerleau [6] who in 1989 built the Autonomous Land Vehicle in a Neural Network (ALVINN) system. Training with data from only the human driver is not sufficient. The network must learn how to recover from mistakes. ...”

End to End Learning for Self-Driving Cars , Bojarski et al. 2016

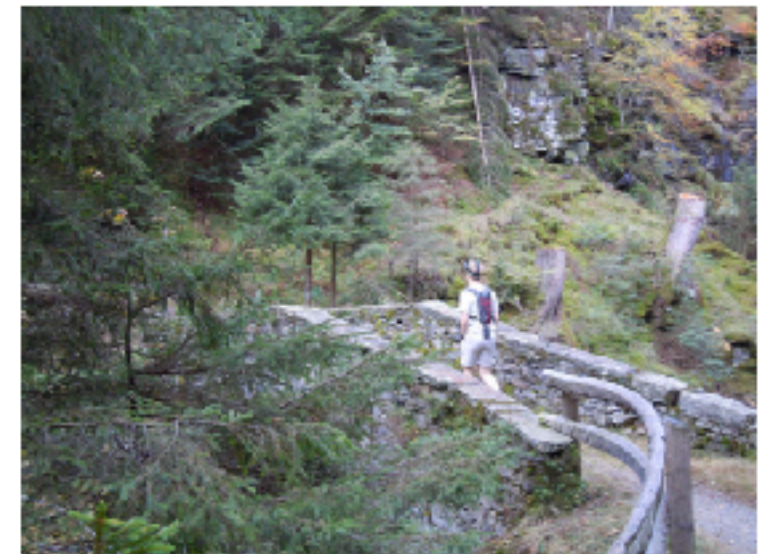
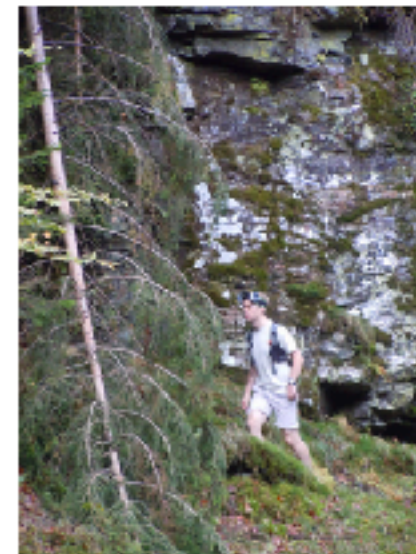
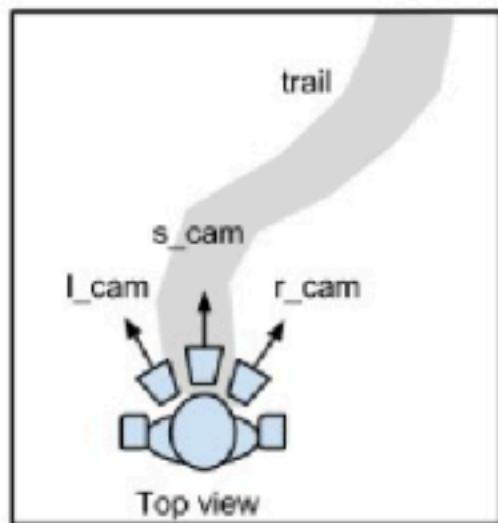
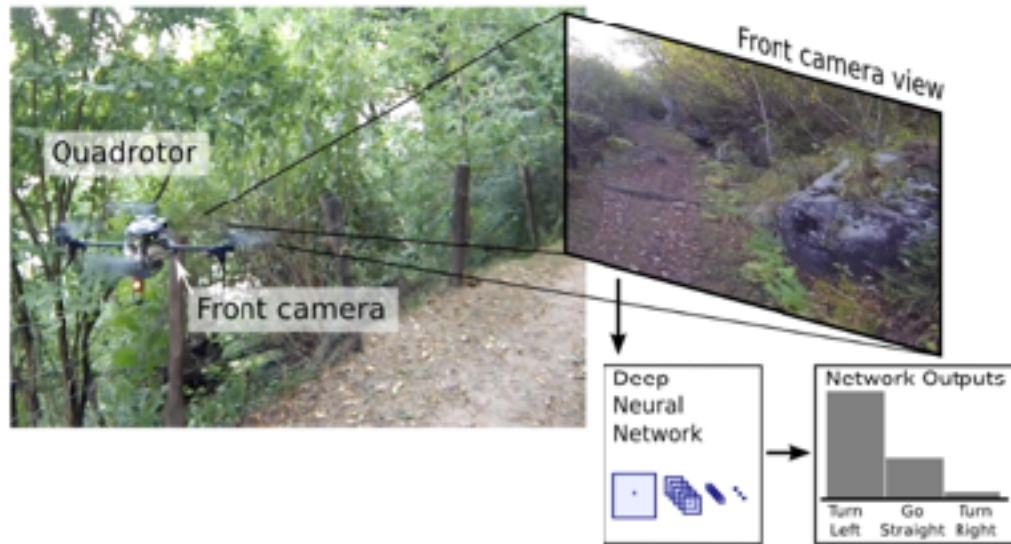
Data Augmentation (2): NVIDIA 2016

DAVE 2 Driving a Lincoln

- A convolutional neural network
- Trained by human drivers
- Learns perception, path planning, and control
"pixel in, action out"
- Front-facing camera is the only sensor

"DAVE-2 was inspired by the pioneering work of Pomerleau [6] who in 1989 built the Autonomous Land Vehicle in a Neural Network (ALVINN) system. Training with data from only the human driver is not sufficient. The network must learn how to recover from mistakes. ...", End to End Learning for Self-Driving Cars , Bojarski et al. 2016

Data Augmentation (3): Trails 2015



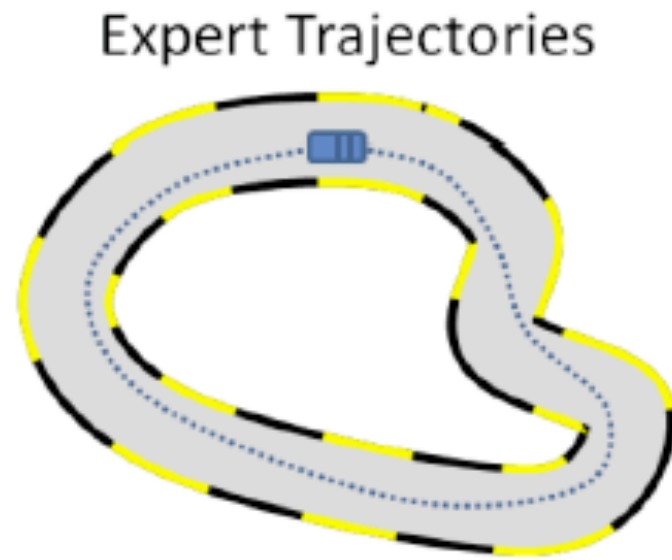
Solution: data augmentations

Change $P_{\pi^*}(\mathbf{o}_t)$ by **augmenting** the expert demonstration trajectories.

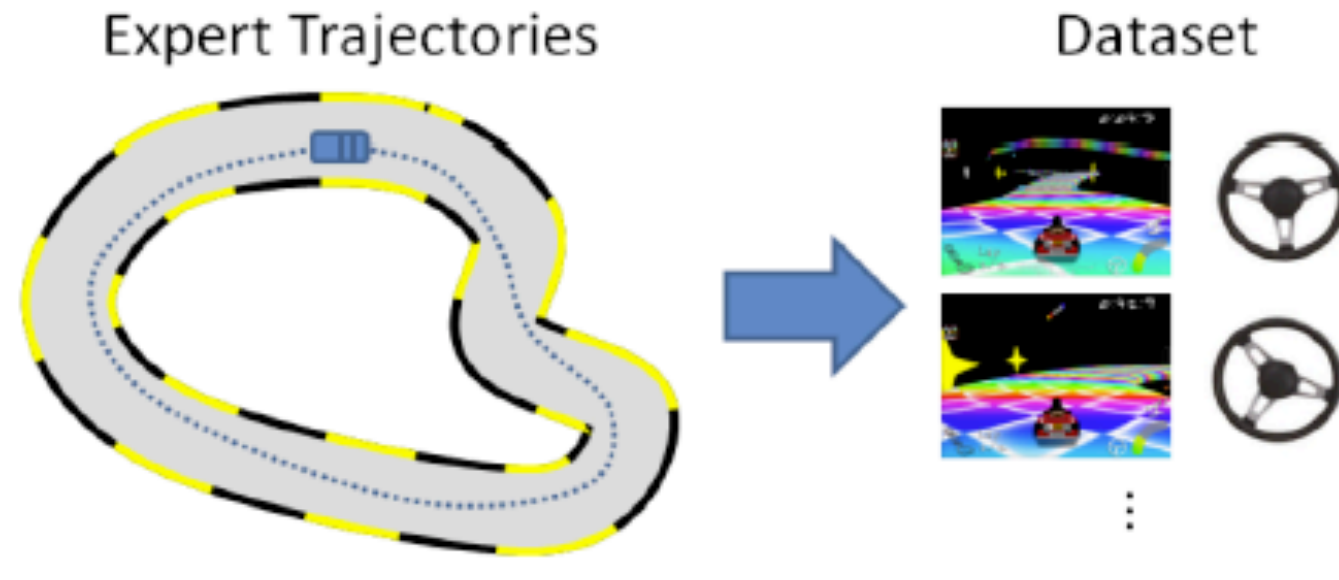
Add examples in expert demonstration trajectories to cover the states/ observations points where the agent will land when trying out its own policy. How?

1. By generating synthetic data in simulation
2. By collecting additional data via clever hardware
3. By interactively querying the experts in additional datapoints

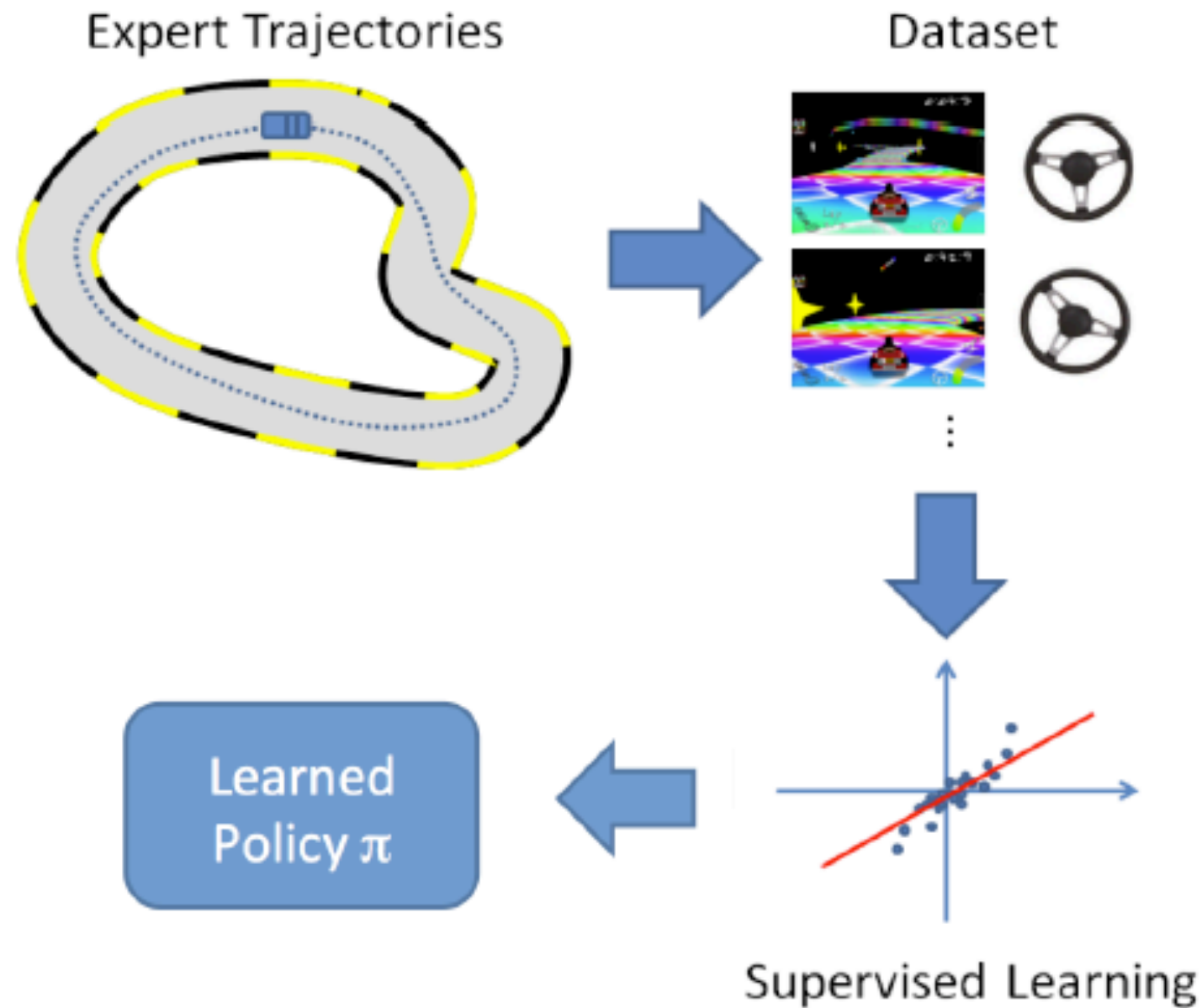
Learning to Drive a Car: Supervised Learning



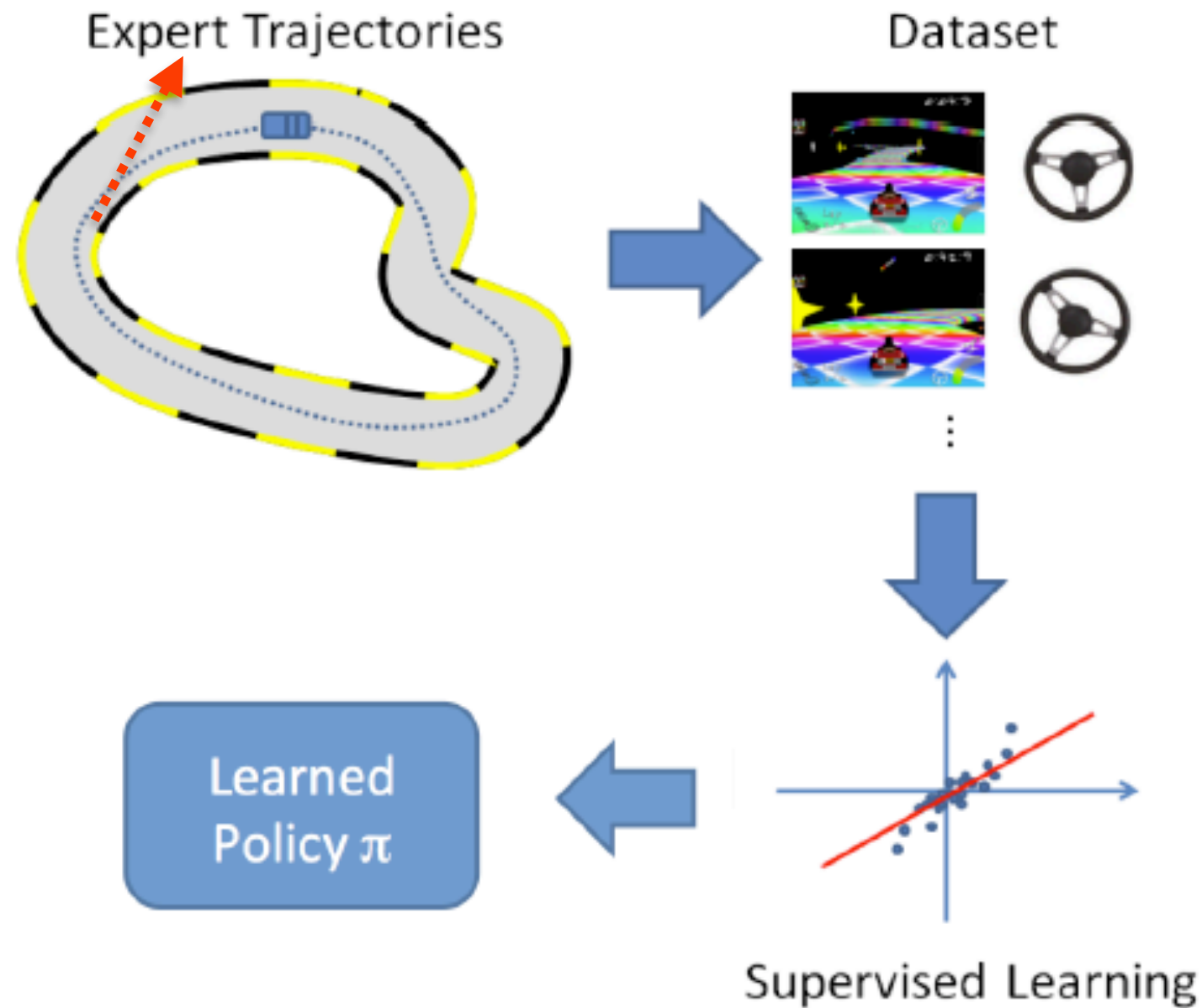
Learning to Drive a Car: Supervised Learning



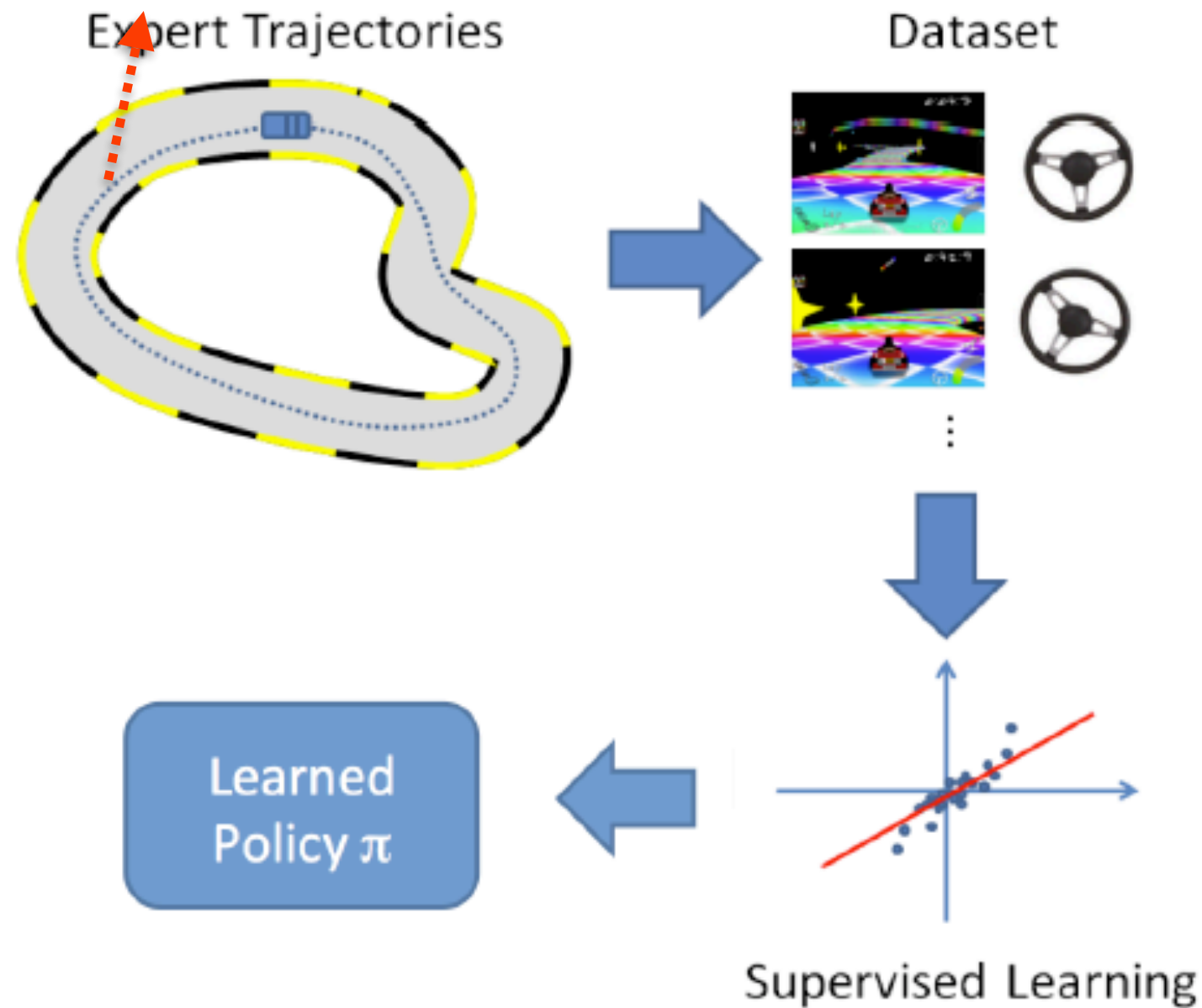
Learning to Drive a Car: Supervised Learning



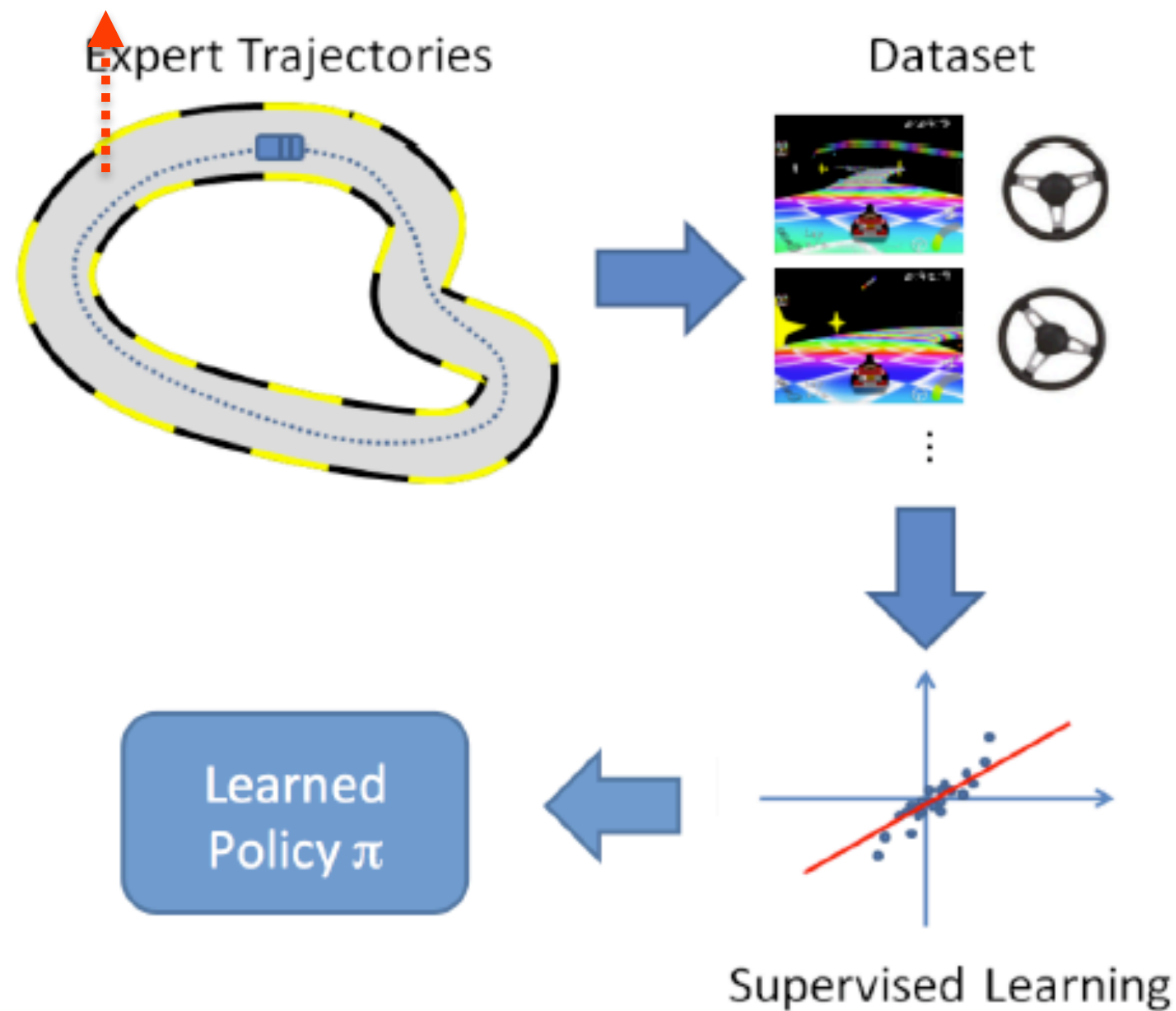
Learning to Drive a Car: Supervised Learning



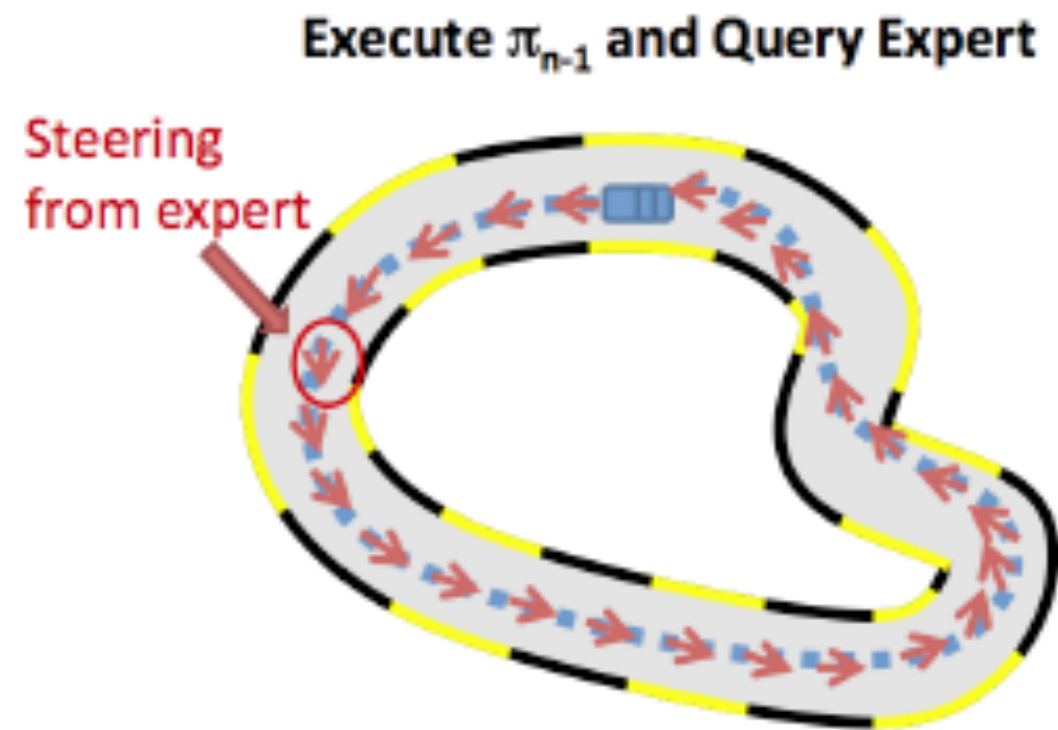
Learning to Drive a Car: Supervised Learning



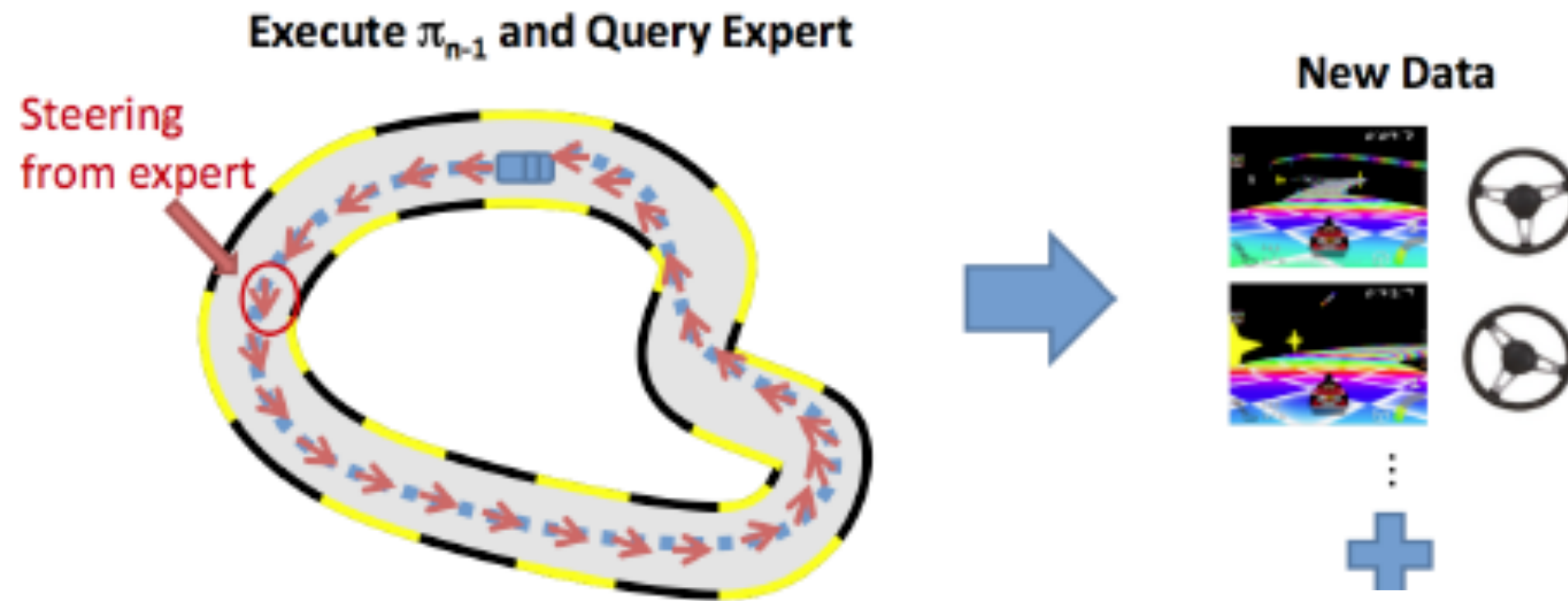
Learning to Drive a Car: Supervised Learning



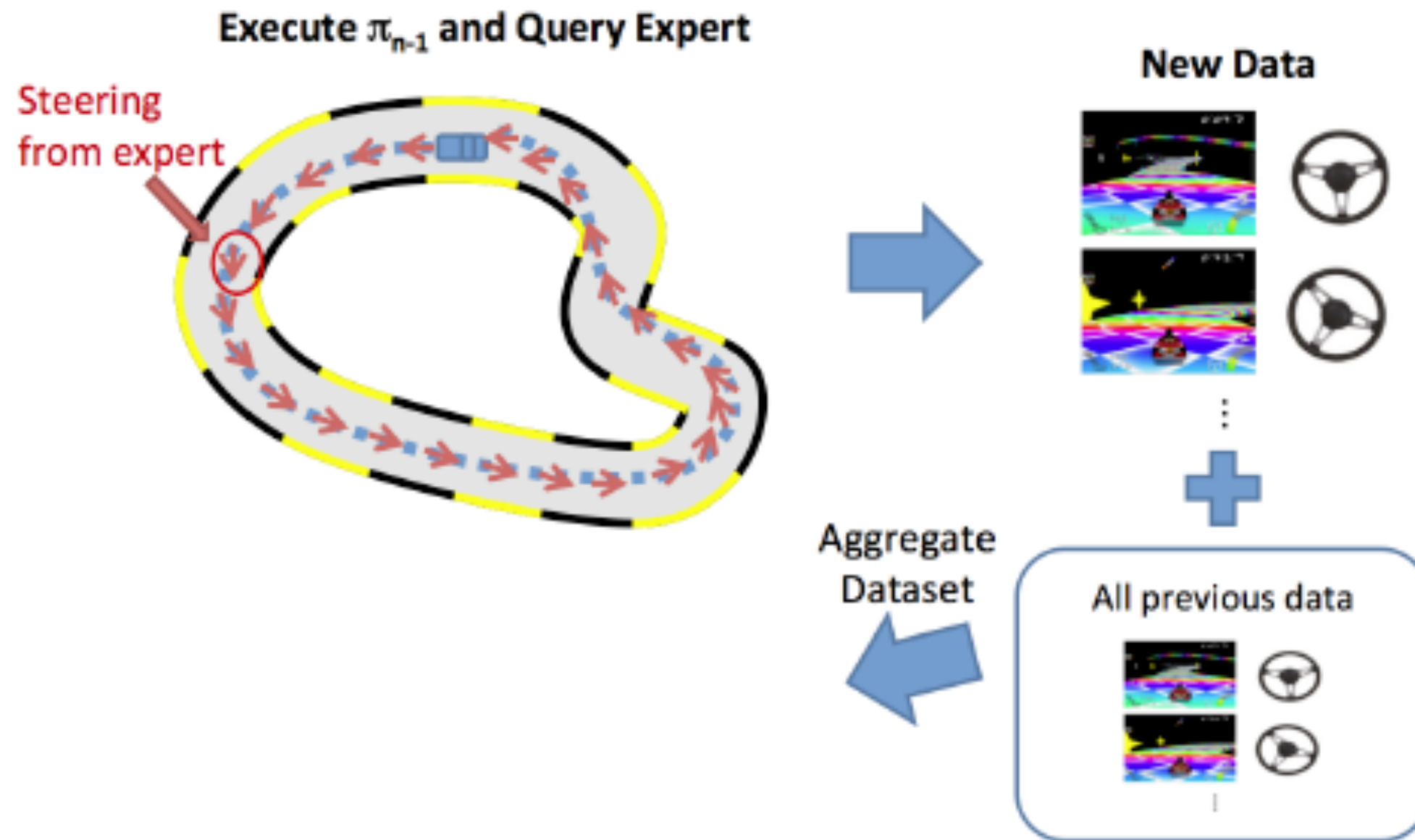
Learning to Race a Car : Interactive learning-DAGGer



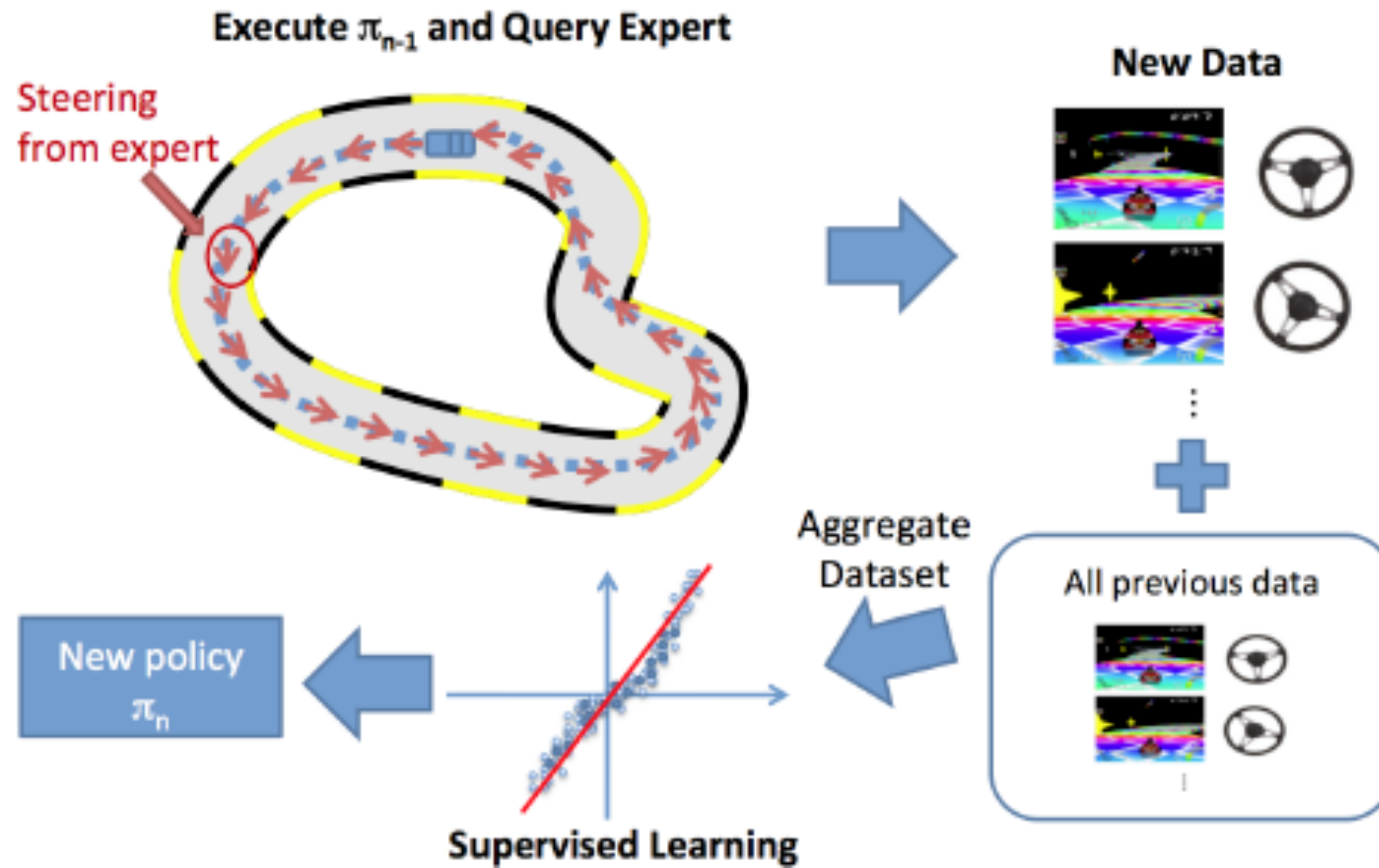
Learning to Race a Car : Interactive learning-DAGGer



Learning to Race a Car : Interactive learning-DAGGer

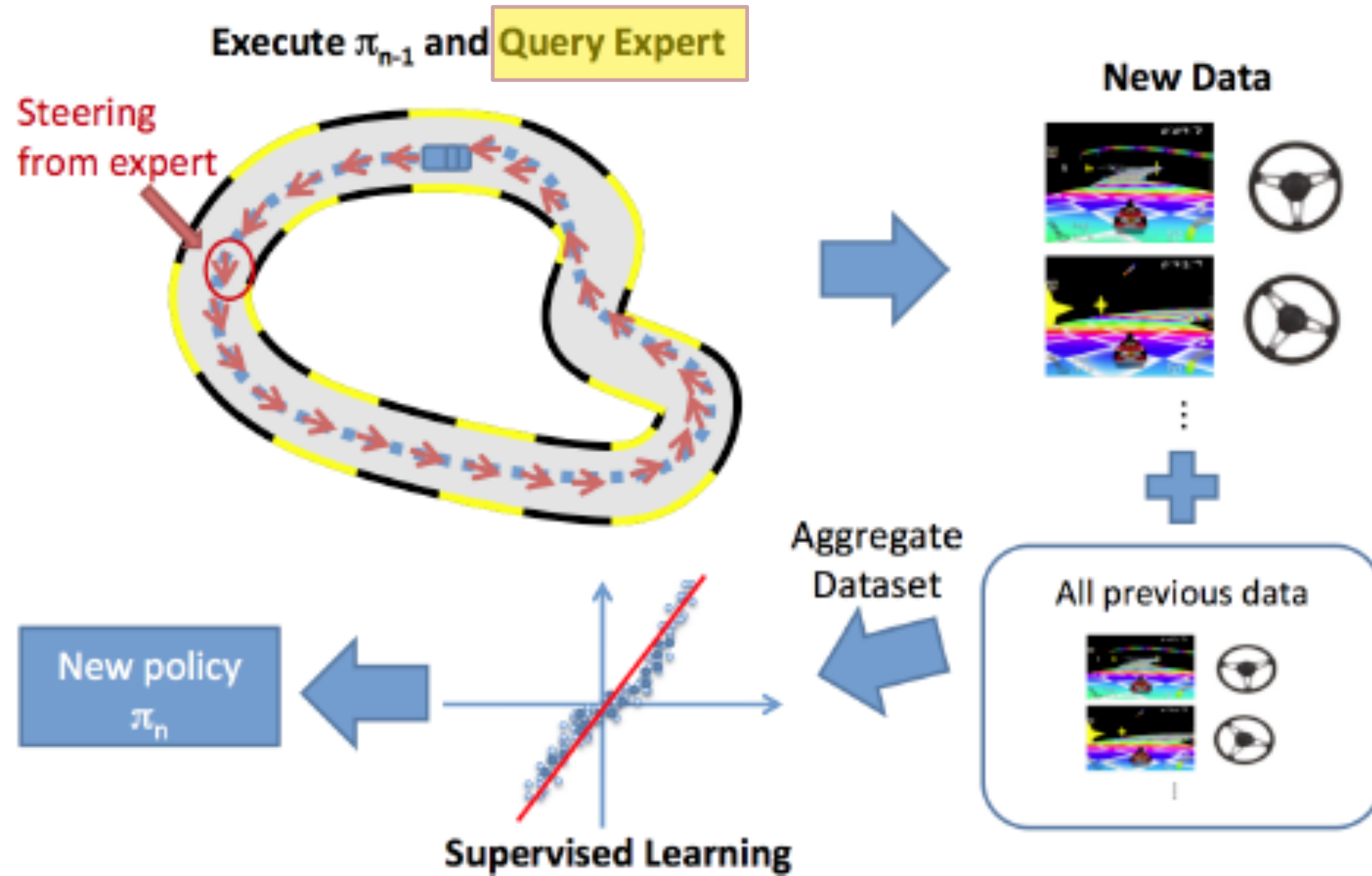


Learning to Race a Car : Interactive learning-DAGGer



Learning to Race a Car : Interactive learning-DAGGer

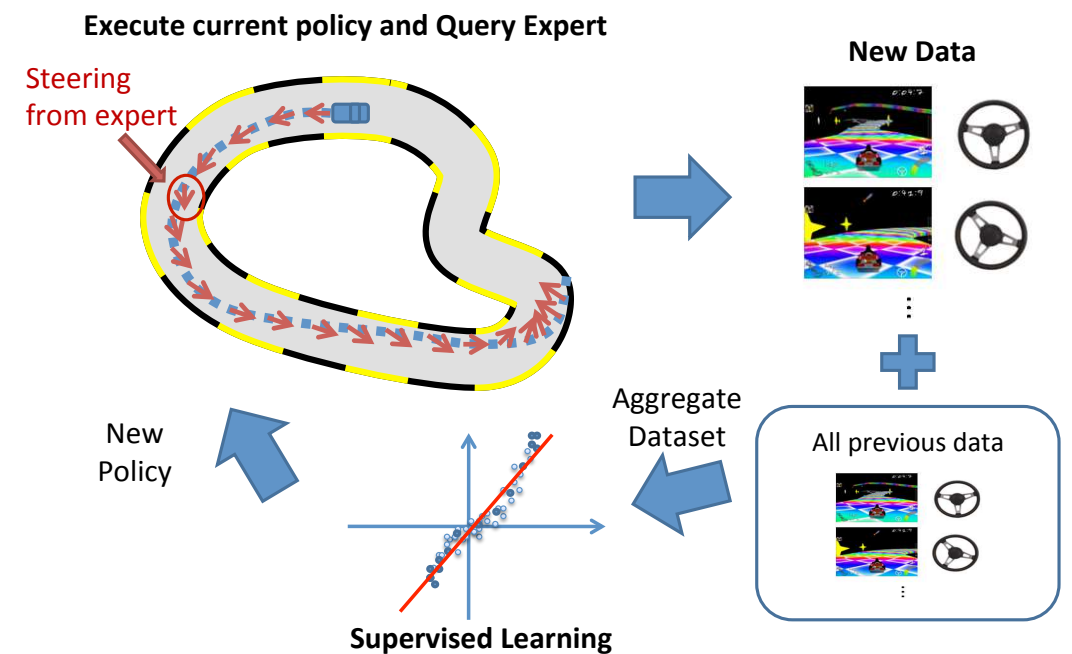
This assumes you can actively access an expert during training!



A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning
Stephane Ross, Geoffrey J. Gordon, J. Andrew Bagnell

DAGGER (in simulation)

Dataset AGGregation: bring learner's and expert's trajectory distributions closer by labelling additional data points resulting from applying the current policy.



A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning, *Ross et al. 2011*

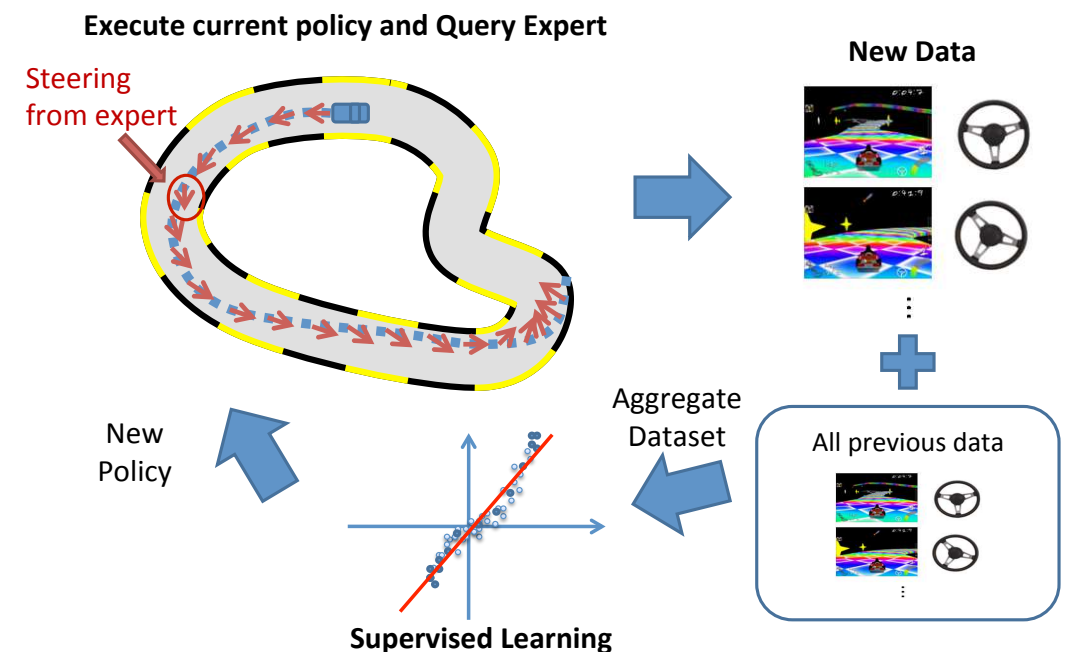
DAGGER (in simulation)

Dataset AGGregation: bring learner's and expert's trajectory distributions closer by (asking human experts to provide) labelling additional data points resulting from applying the current policy

1. Train $\pi_{\theta}(u_t | o_t)$ from human data $\mathcal{D}_{\pi^*} = \{o_1, u_1, \dots, o_N, u_N\}$
2. Run $\pi_{\theta}(u_t | o_t)$ to get dataset $\mathcal{D}_{\pi} = \{o_1, \dots, o_M\}$
3. Ask human to label $\mathcal{D}_{\pi}$ with actions u_t
4. Aggregate: $\mathcal{D}_{\pi^*} \leftarrow \mathcal{D}_{\pi^*} \cup \mathcal{D}_{\pi}$
5. GOTO step 1.

Problems:

- execute an unsafe/partially trained policy
- repeatedly query the expert



DAGGER (on a real platform)

Application on drones: given RGB from the drone camera predict steering angles

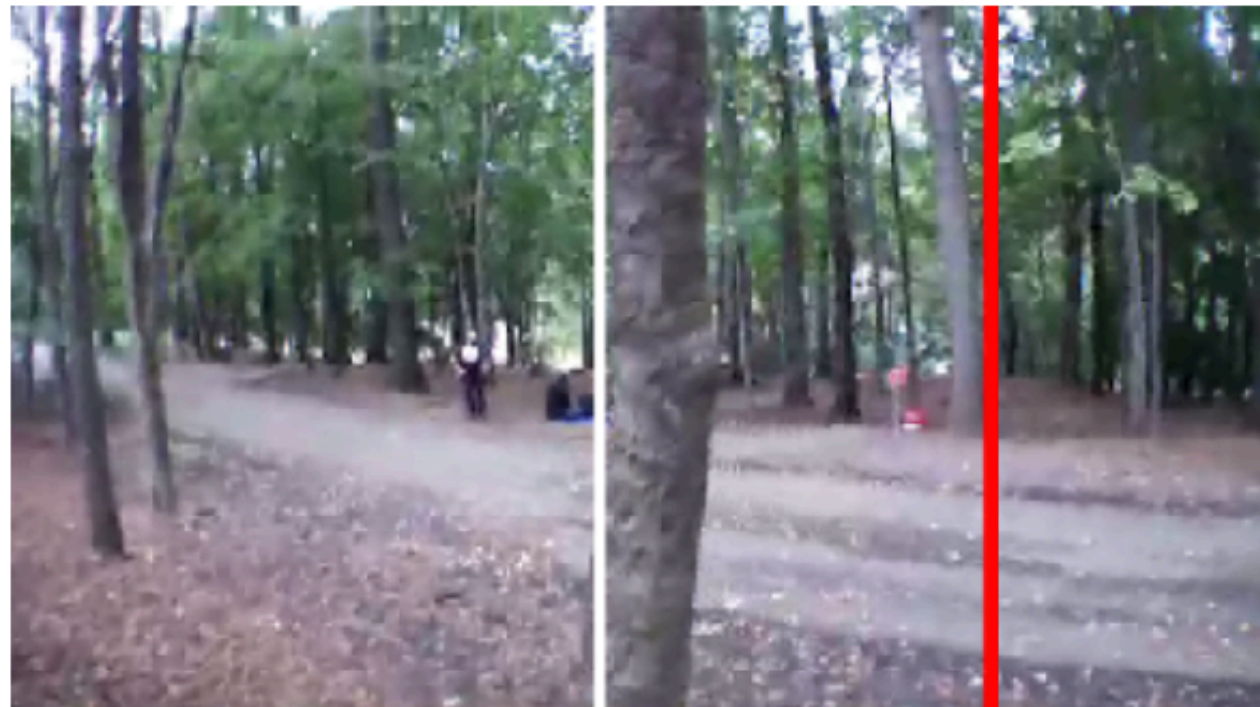


DAGGER (on a real platform)

Application on drones : given RGB from the drone camera predict steering angle

Caveats:

1. It is hard for the expert to provide the right magnitude for the turn without feedback of his own actions! Solution: provide him with visual feedback



DAGGER (on a real platform)

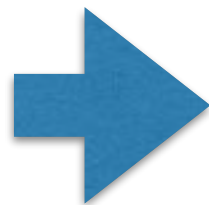
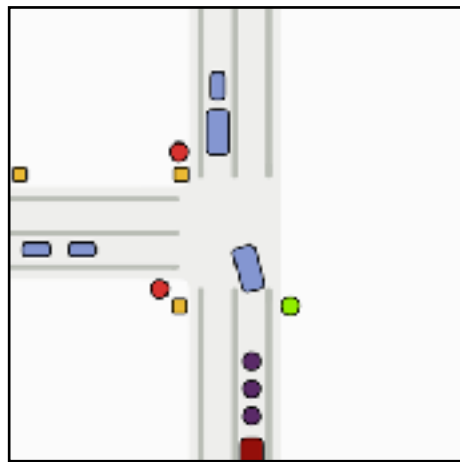
Caveats:

1. Is hard for the expert to provide the right magnitude for the turn **without feedback of his own actions!** Solution: provide him with his visual feedback
 2. The expert's **reaction time** to the drone's behavior is **large**, this causes imperfect actions to be commanded. Solution: play-back in slow motion offline and record their actions.
 3. Executing an **imperfect policy causes accidents**, crashes into obstacles. Solution: safety measures which make again the data distribution matching imperfect between train and test, but good enough.
- **Experts do not need to be humans.**
 - **Machinery that we develop in this lecture can be used for imitating expert policies found through (easier) optimization in a constrained smaller part of the state space.**
 - **Imitation then means distilling knowledge of expert constrained policies into a general policy that can do well in all scenarios the simpler policies do well.**

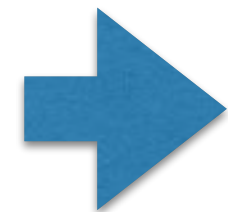
Learning by Cheating

DAGGER: from a privileged teaching agent to an agent that drives from images.

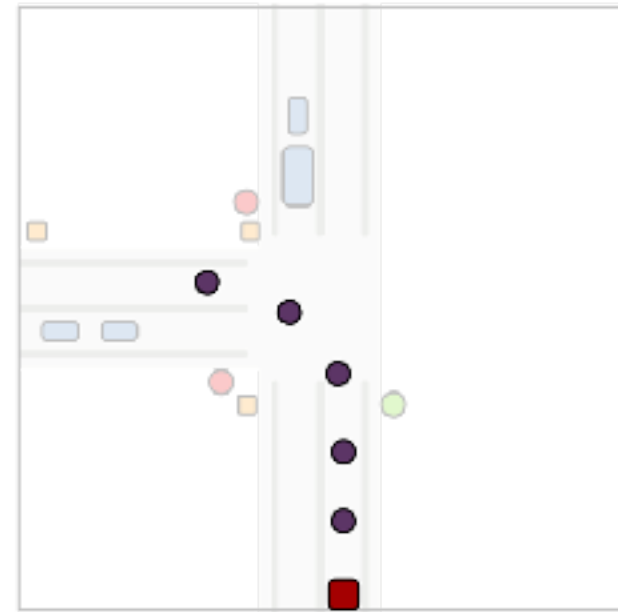
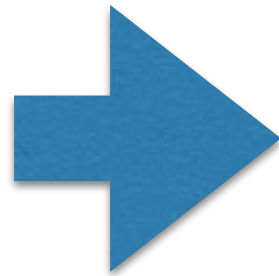
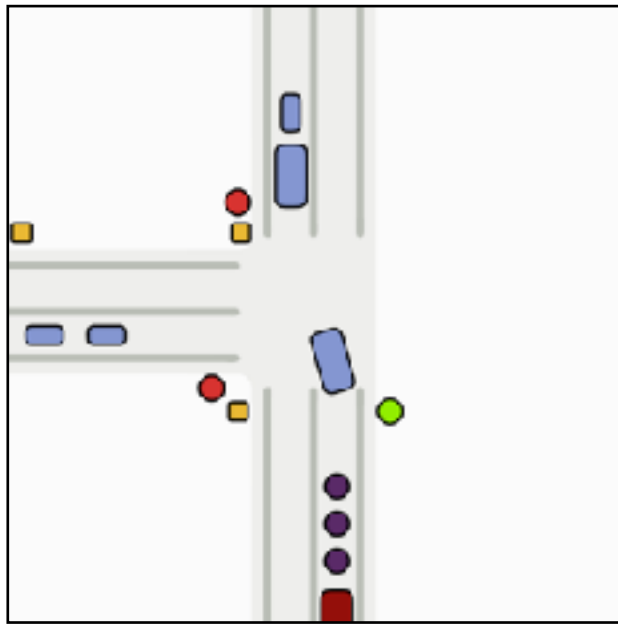
privileged agent



sensorimotor agent

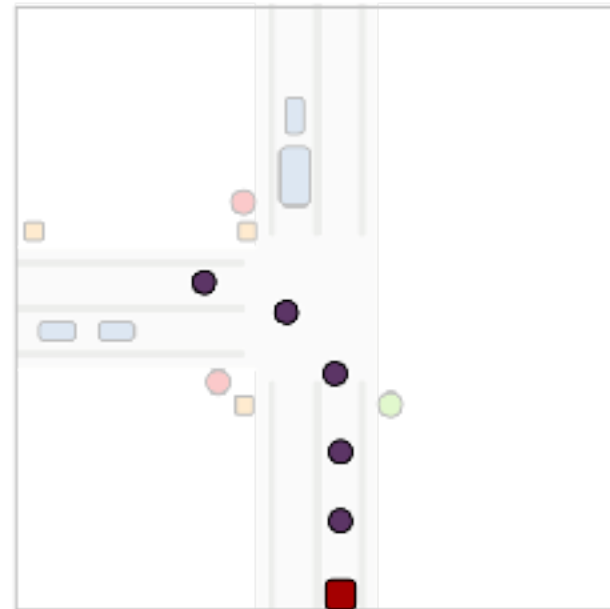
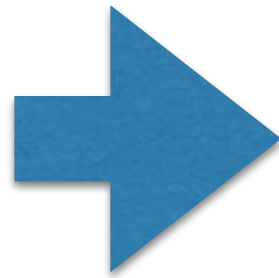


Privileged Agent cheats: drives with the internal state of the simulator



it predicts future waypoints for the car to follow

Sensorimotor Agent drives from images



it predicts future waypoints for the car to follow.

Problem: It is hard to imagine how the scene would look like from alternative view points, to be able to perform DAGGER.

Waypoints are translated to steering commands with a low-level controller

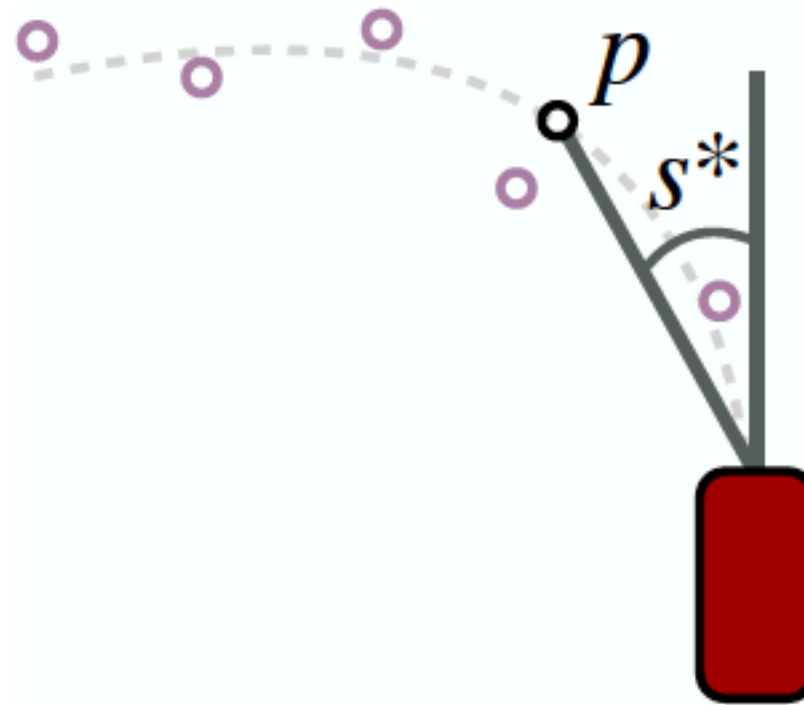
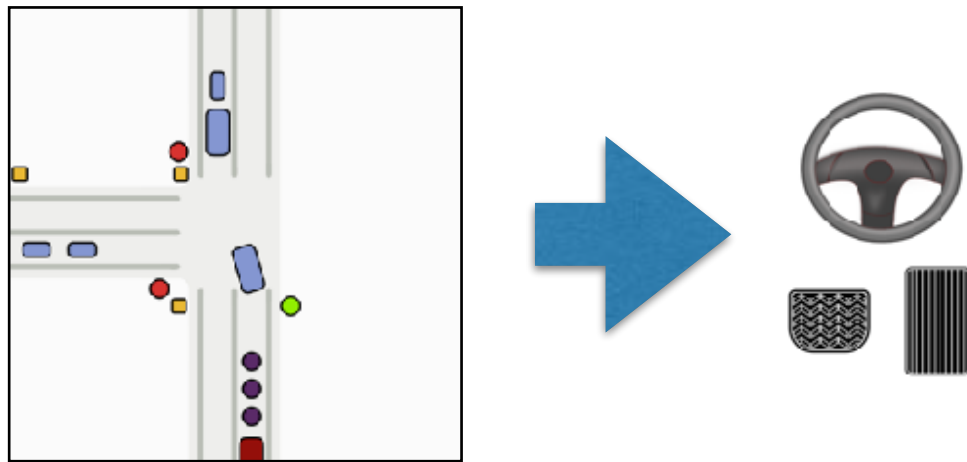


Figure 4: Lateral PID controller. Here the agent aims at the projection of the second waypoint onto the fitted arc. s^* denotes the angle between the vehicle and the target point p .

Learning by Cheating

DAGGER: from a privileged teaching agent to an agent that drives from images.

privileged agent



sensorimotor agent



trained with imitation learning
from human experts

47

trained with imitation learning
from the privileged agent

But why learning from simplified input helps?

It is the same supervised learning problem!

The privileged agent can generate augmentations for the other one!

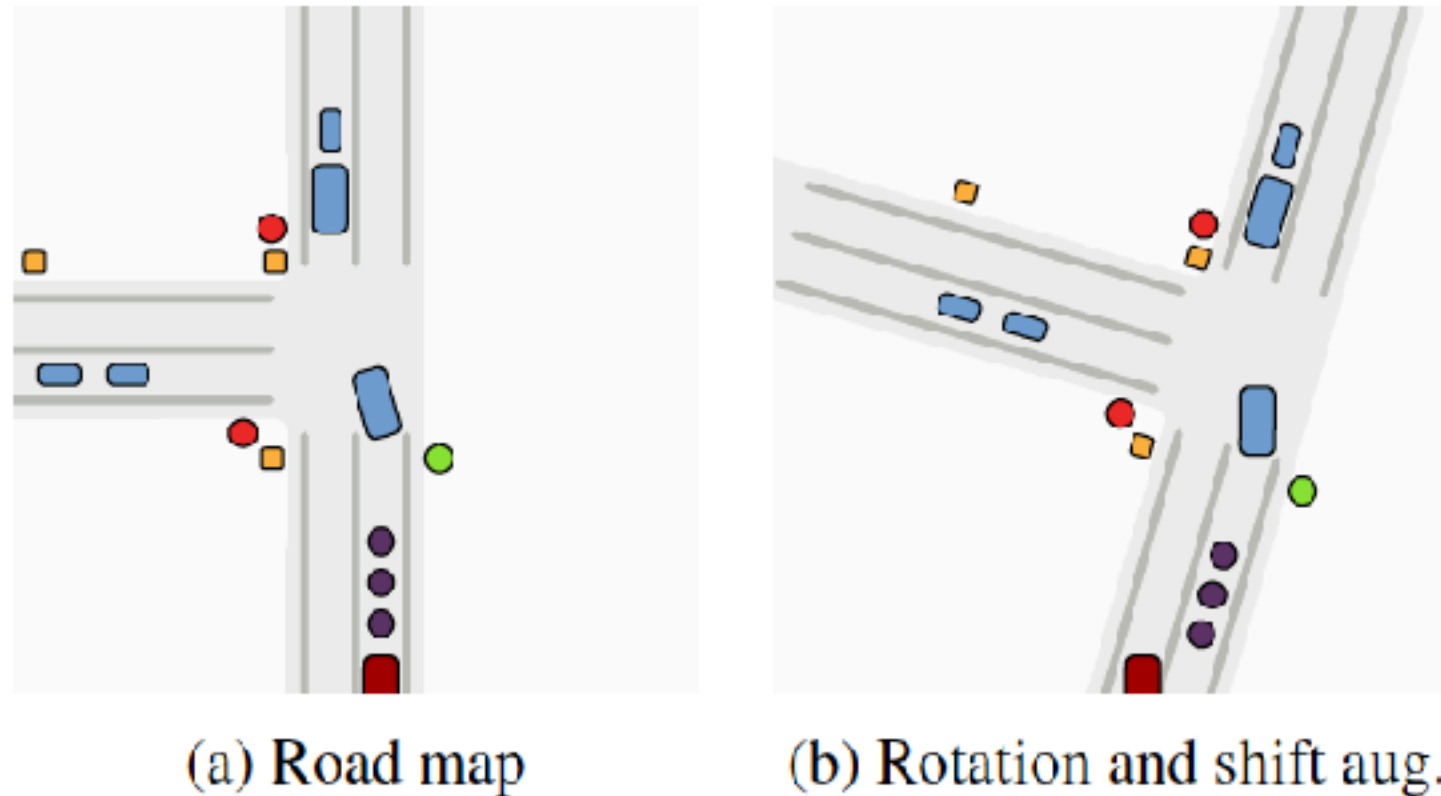


Figure 3: **(a)** Map M provided to the privileged agent. One channel each for road (light grey), lane boundaries (grey), vehicles (blue), pedestrians (orange), and traffic lights (green, yellow, and red). The agent is centered at the bottom of the map. The agent's vehicle (dark red) and predicted waypoints (purple) are shown for visualization only and are not provided to the network. **(b)** The map representation affords simple and effective data augmentation via rotation and shifting.

Results

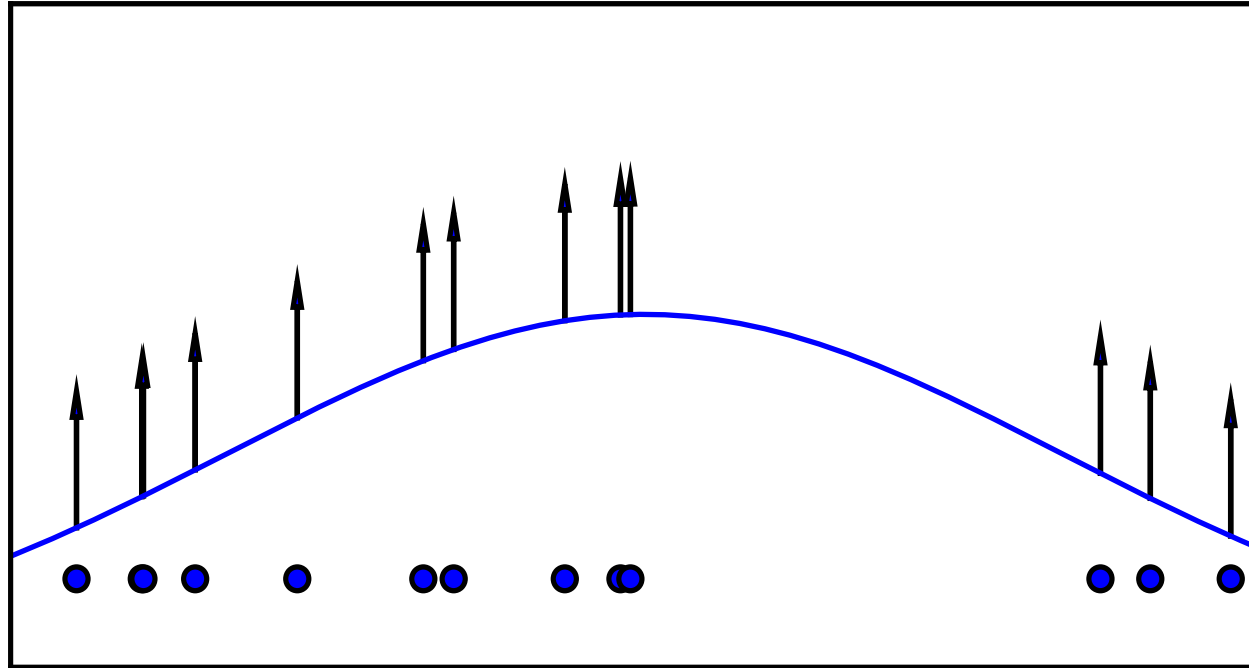


- high level command
- controller error
- lack of temporal reasoning

Challenges in Imitation Learning

- State distribution shift between train and test sets
- Multi-modality of state-conditioned action distributions
- Generalization (as for all supervised learning problems)

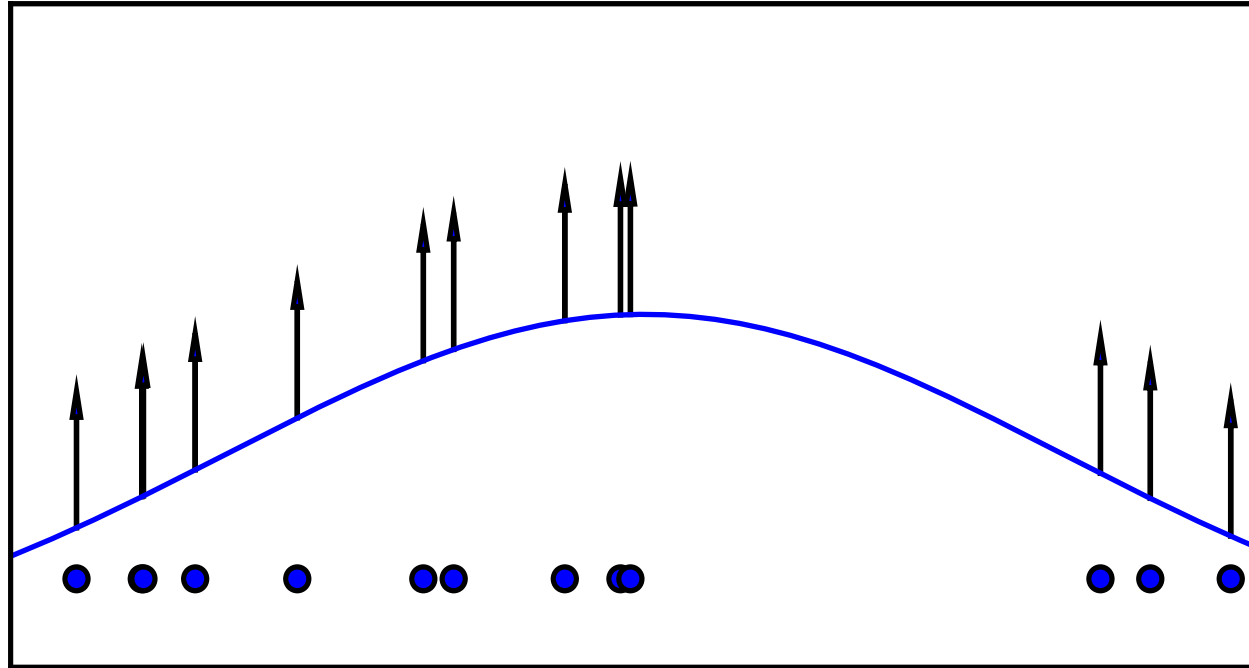
Maximum Likelihood



$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} | \theta)$$

$$\theta^* = \arg \max_{\theta} \sum_{i=1}^N \log p_{\text{model}}(\mathbf{x}_i | \theta)$$

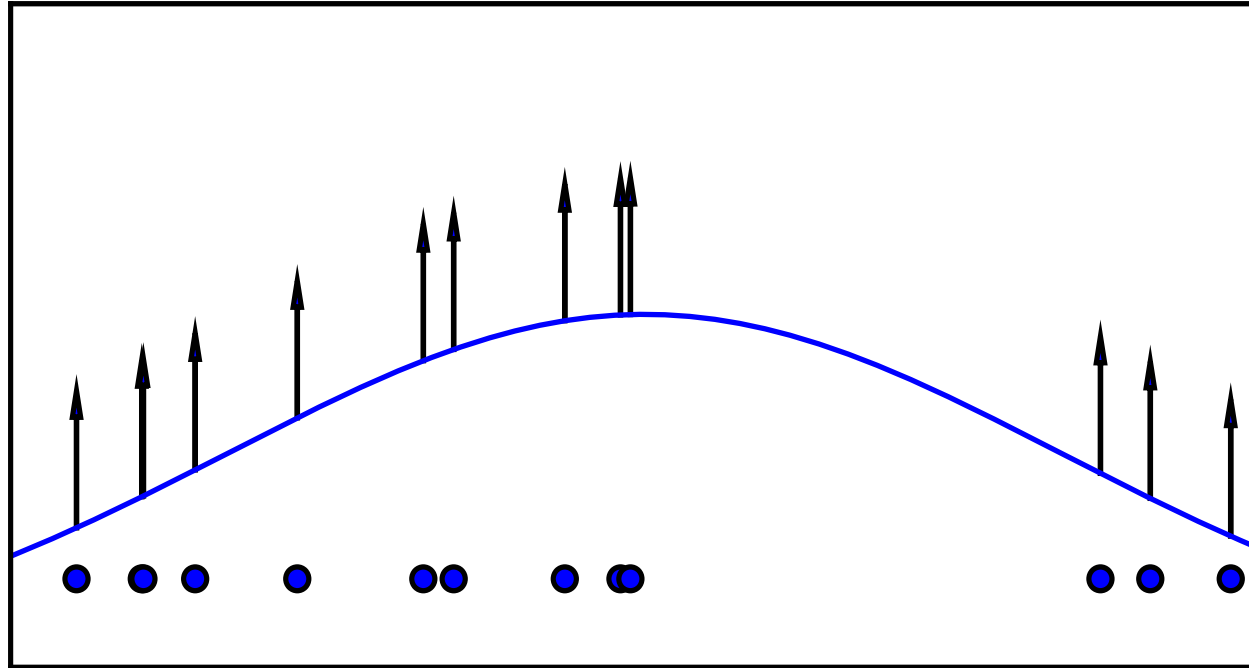
Maximum Likelihood



$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} | \theta)$$

explicit density

Maximum Conditional Likelihood



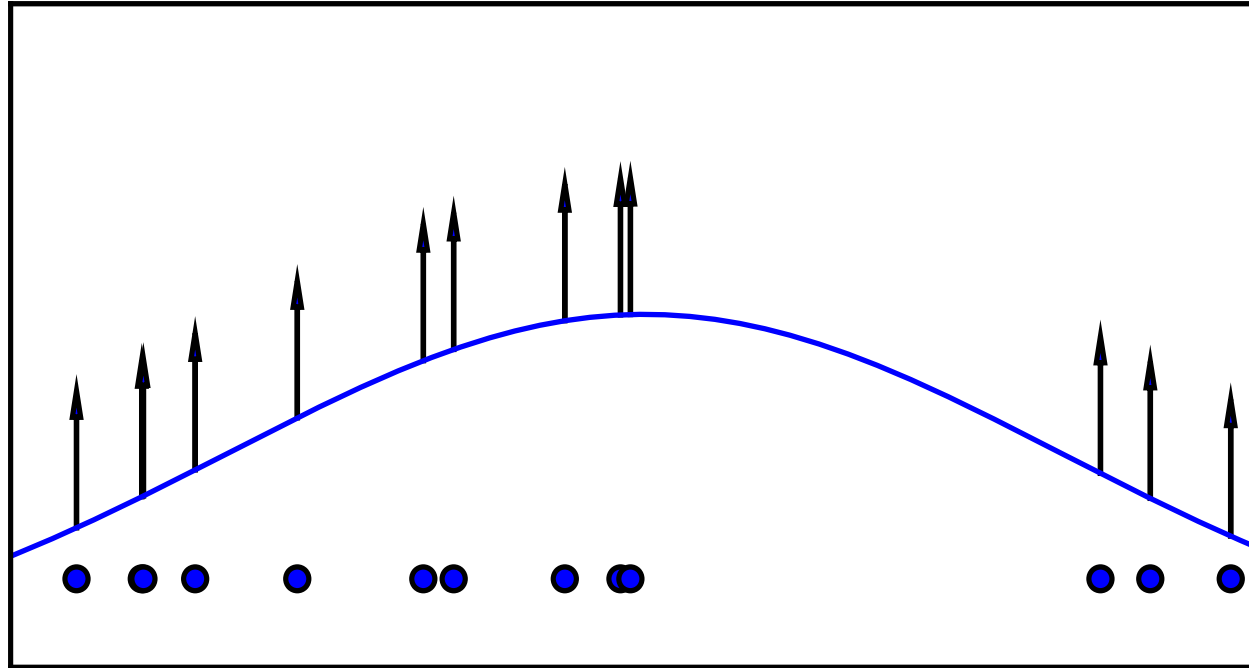
$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} \mid \theta, c)$$

explicit density

extra conditioning information

Maximum Conditional Likelihood

$$D_{\text{KL}}(P\|Q) = - \sum_{x \in X} P(x) \log \left(\frac{Q(x)}{P(x)} \right)$$

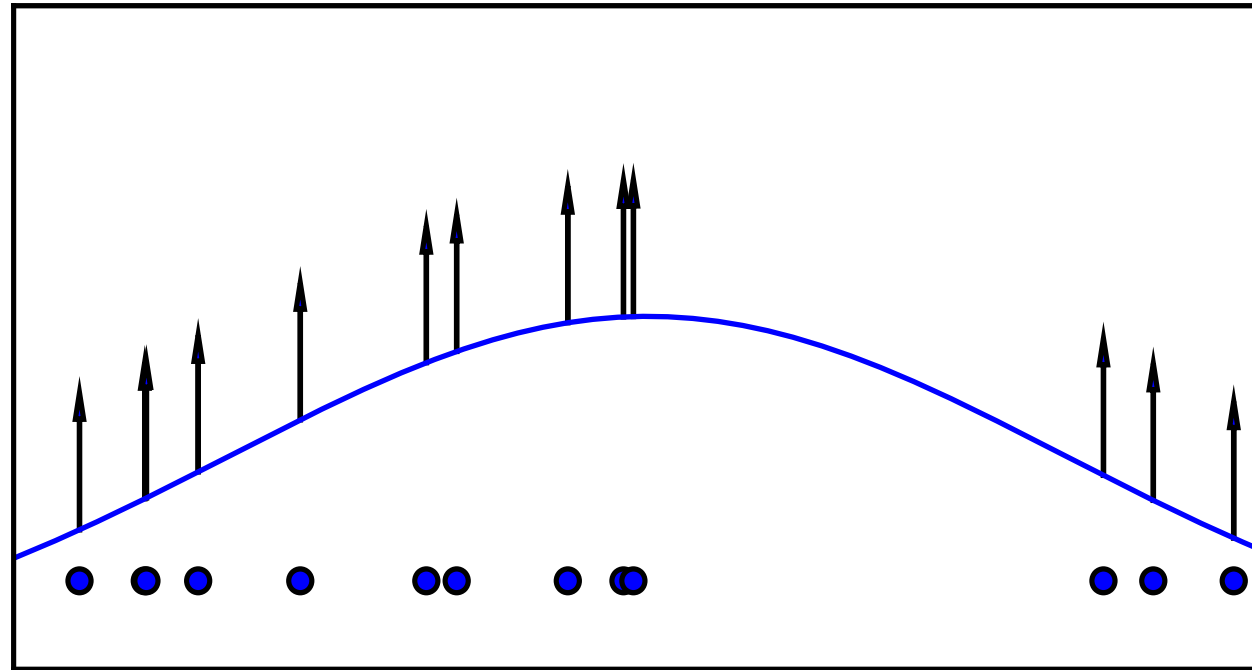


$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} \mid \theta, \mathbf{c})$$

equiv. to

$$\theta^* = \arg \min_{\theta} D_{\text{KL}}(p_{\text{data}} \parallel p_{\text{model}}(\mathbf{x} \mid \theta, \mathbf{c}))$$

Maximum Likelihood-Gaussian with fixed covariance

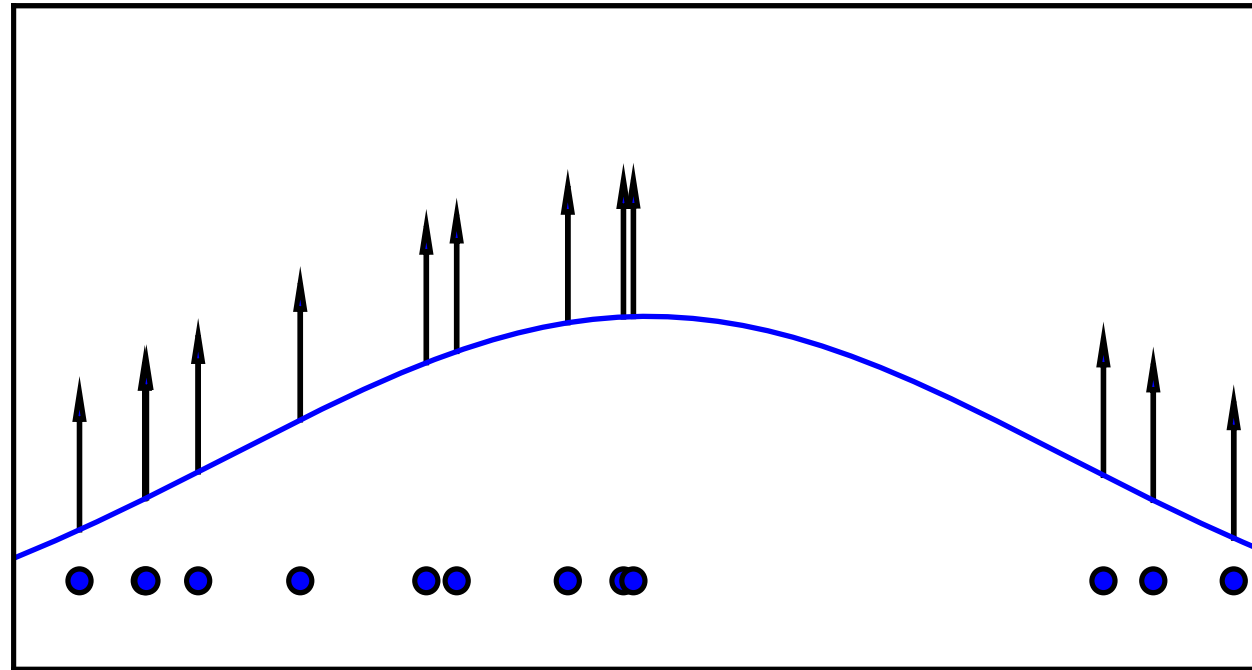


$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} \mid \theta, \mathbf{c})$$

$$p_{\text{model}}(\mathbf{x} \mid \theta, \mathbf{c}) = \frac{1}{(2\pi)^{-\frac{k}{2}} \det(\Sigma)^{-\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu(\theta, \mathbf{c}))^{\top} \Sigma^{-1} (\mathbf{x} - \mu(\theta, \mathbf{c})) \right), \text{ where } \Sigma = \mathbf{I}$$

Maximum Likelihood-Gaussian with fixed covariance

$$p_{\text{model}}(\mathbf{x} | \theta, \mathbf{c}) = \frac{1}{(2\pi)^{-\frac{k}{2}} \det(\Sigma)^{-\frac{1}{2}}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu(\theta, \mathbf{c}))^\top \Sigma^{-1} (\mathbf{x} - \mu(\theta, \mathbf{c})) \right), \text{ where } \Sigma = \mathbf{I}$$



$$\theta^* = \arg \max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} | \theta, \mathbf{c})$$

$$\max_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \log p_{\text{model}}(\mathbf{x} | \theta, \mathbf{c}) \quad \text{equiv. to}$$

$$\min_{\theta} \mathbb{E}_{x \sim p_{\text{data}}} \|\mathbf{x} - \mu(\theta, \mathbf{c})\|_2^2$$

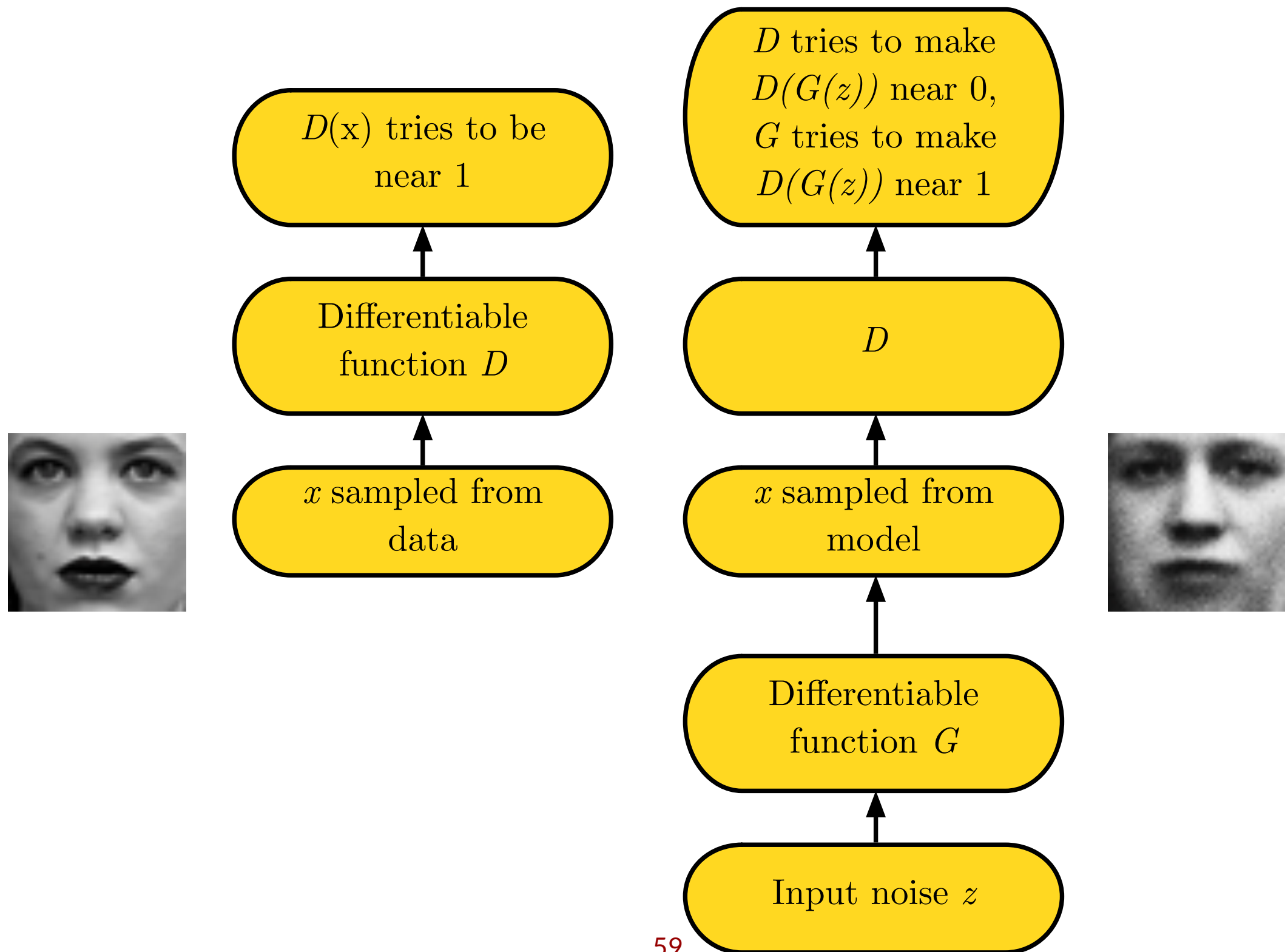
e.g. behavior cloning with continuous actions

$$\mathcal{L}_{BC}(\theta, \mathcal{T}) = \mathbb{E}_{(s_t^j, a_t^j) \sim \mathcal{T}} \left[\|a_t^j - \pi_{\theta}(s_t^j)\|_2^2 \right]$$

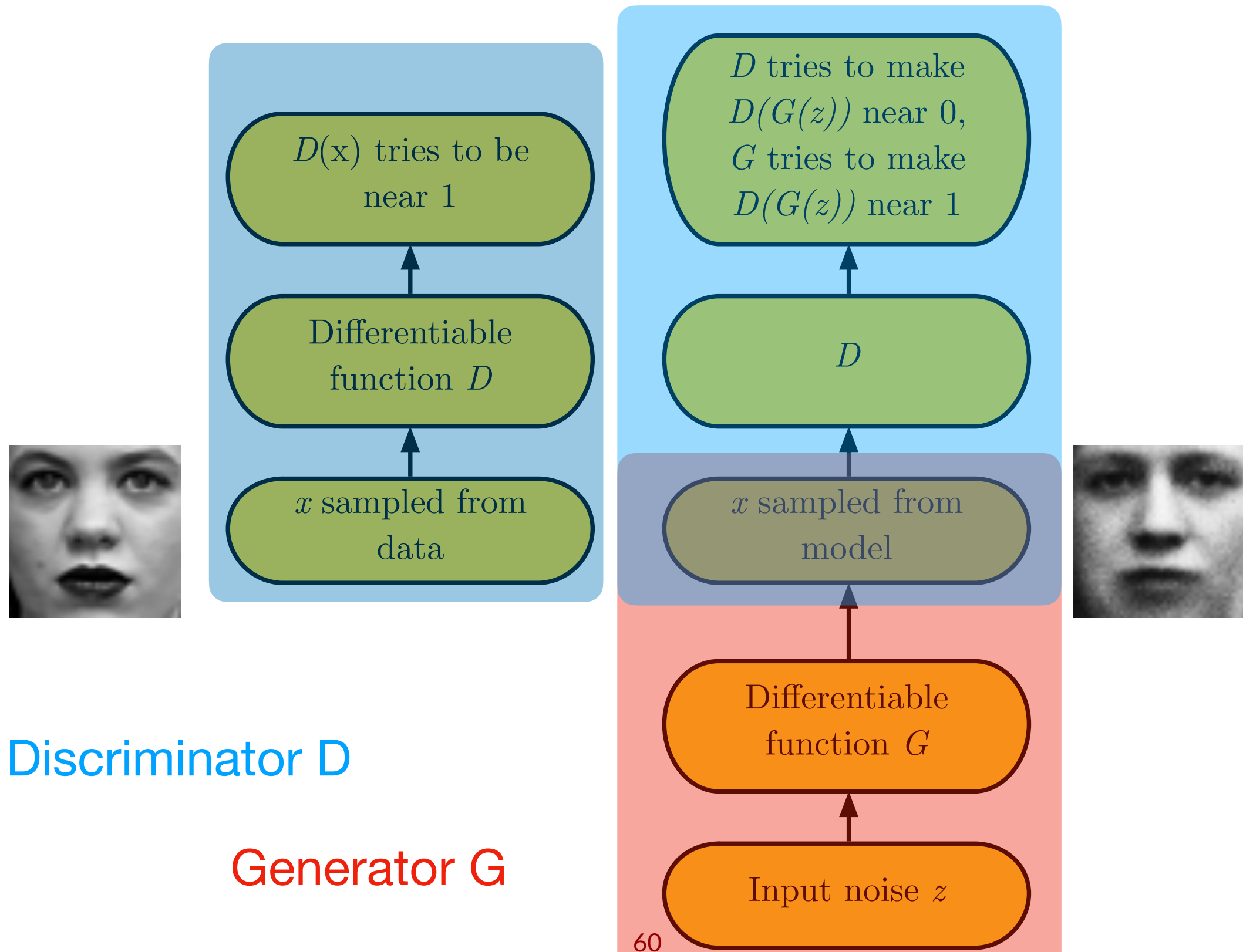
What about using more expressive action distributions?

1. **Generative adversarial networks** for matching the state-action distributions between expert and student
-> solution to state distribution mismatch.
2. **Diffusion models** for matching the state-conditioned action distributions between expert and student

Adversarial Nets Framework



$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$



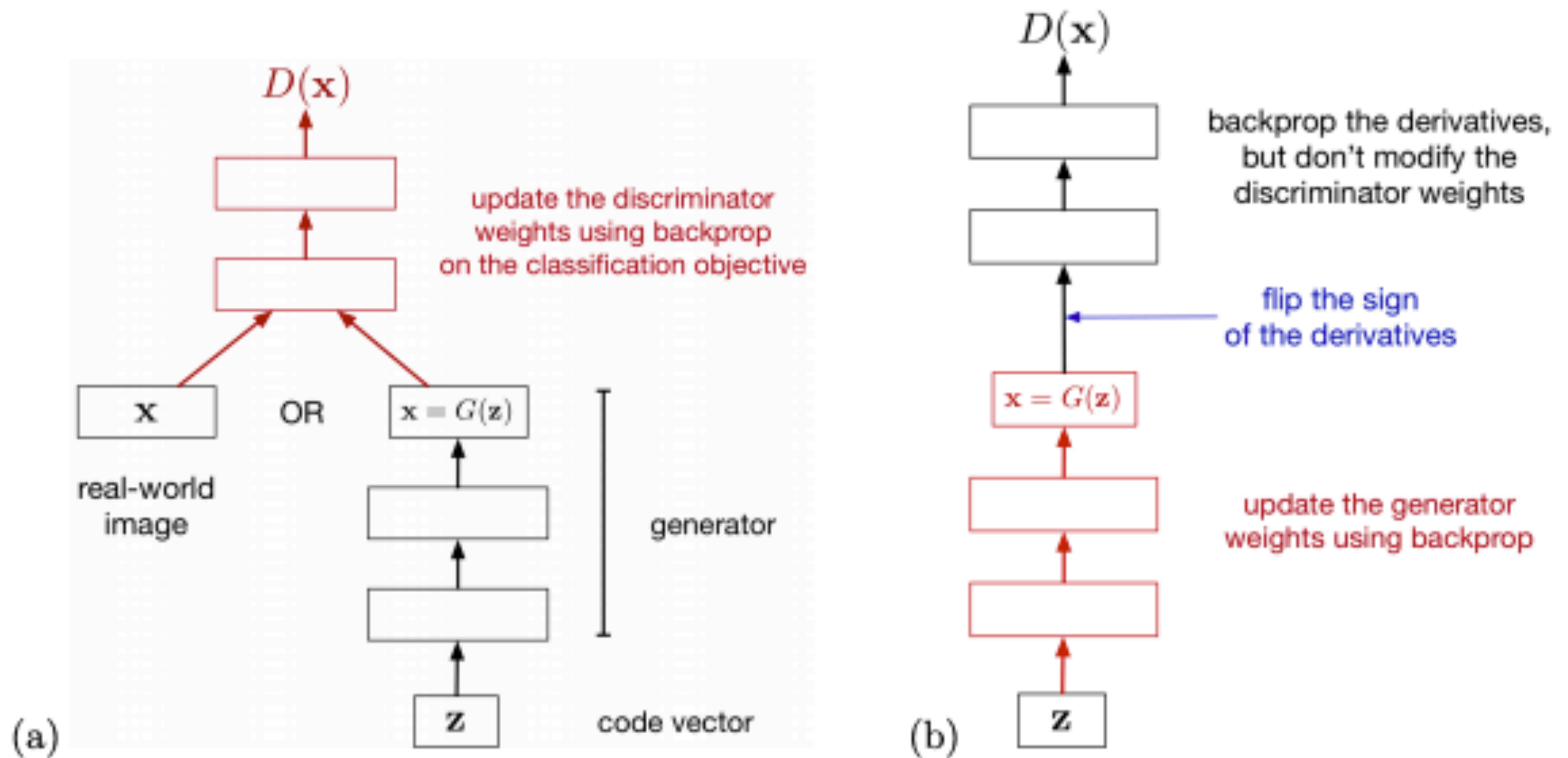
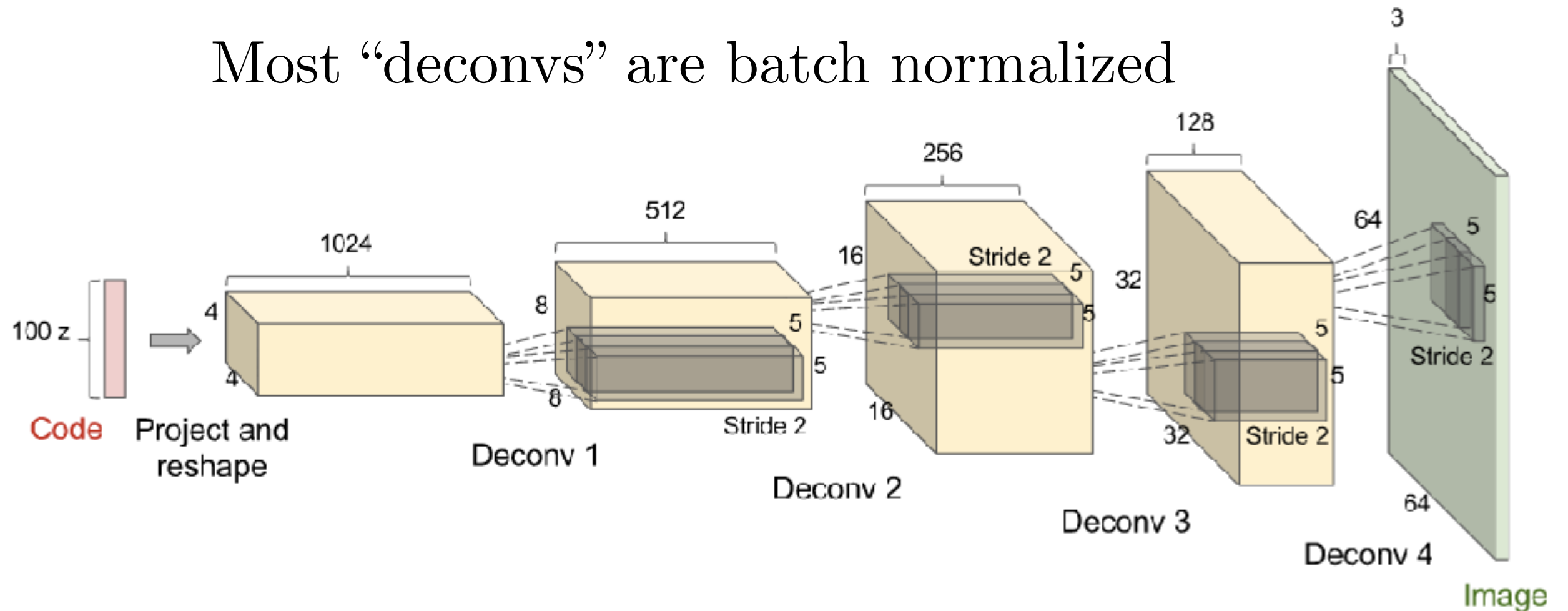


Figure 3: (a) Updating the discriminator. (b) Updating the generator.

A Generator network (DCGAN)

Most “deconvs” are batch normalized

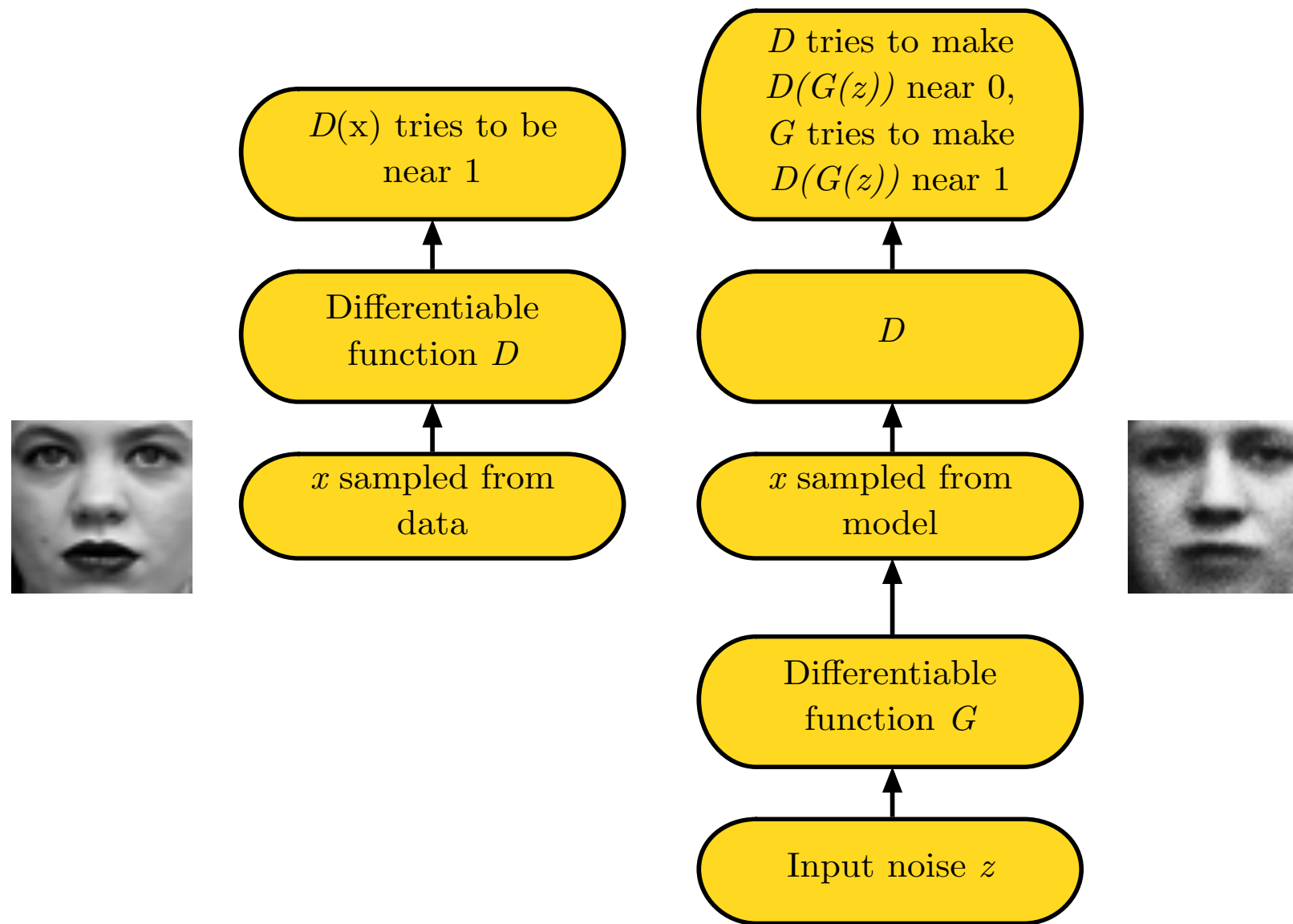


(Radford et al 2015)

(Goodfellow 2016)

Training Procedure

- Use SGD-like algorithm of choice (Adam) on two minibatches simultaneously:
 - A minibatch of training examples
 - A minibatch of generated samples
- Optional: run k steps of one player for every step of the other player.



(Goodfellow 2016)

Questions:

- What if the generator maps all noise vectors to a single super photorealistic image?
- What if we train the discriminator till convergence (it is just a supervised classifier...) and becomes perfect in distinguishing real from generated images?

A minimax game

$$\min_{\color{red}{G}} \max_{\color{blue}{D}} V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log \color{blue}{D}(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - \color{blue}{D}(\color{red}{G}(z)))]$$

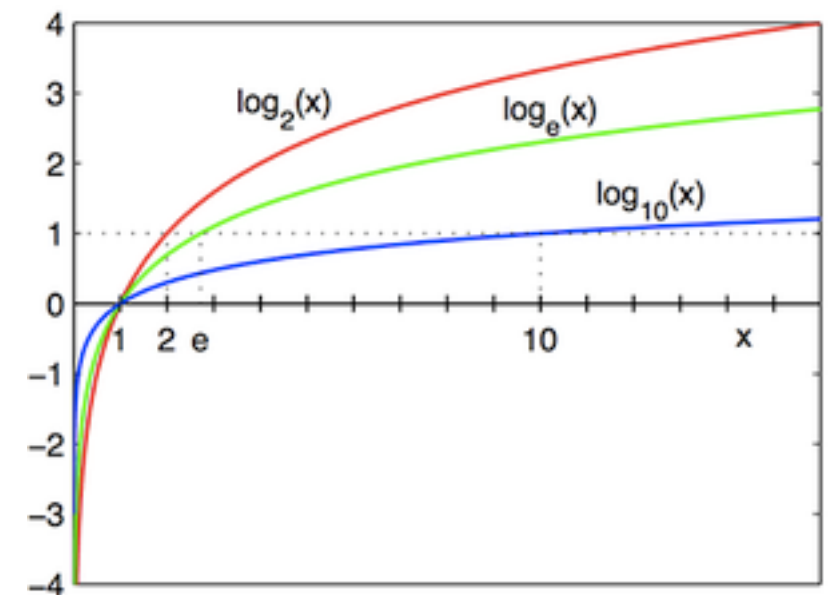
A better cost function

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$$\min_G \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Gradients not informative
when D close to 0

$$\max_G \mathbb{E}_{z \sim p_z(z)} [\log(D(G(z)))]$$



$$\max_D \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

$$\min_D \mathbb{E}_{x \sim p_{data}(x)} [\log(1 - D(x))] + \mathbb{E}_{z \sim p_z(z)} [\log(D(G(z)))]$$

Generative Adversarial Imitation learning

The policy network will be our generator, that conditions on the state:

$$\pi_{\theta}(s) \rightarrow a$$

Generative Adversarial Imitation learning

Find a policy π_θ that makes it impossible for a discriminator network to distinguish between state-action pairs from the expert demonstrations and state-action pairs visited by the agent's policy π_θ :

$$\min_{\pi_\theta} \mathbb{E}_{(s,a) \sim \pi_\theta} [-\log(D_\phi(s, a))]$$

$$\min_{D_\phi} \mathbb{E}_{(s,a) \sim \text{Demo}} [\log(1 - D_\phi(s, a))] + \mathbb{E}_{(s,a) \sim \pi_\theta} [\log(D_\phi(s, a))]$$

The reward for the policy optimization is how well I matched the demonstrator's trajectory distribution, else, how well I confused the discriminator.

$$r(s, a) = \log D_\phi(s, a), (s, a) \sim \pi_\theta$$

Generative Adversarial Imitation learning

Input: Expert trajectories , initial policy parameters θ_0 and initial discriminator weights ϕ_0 .

For $i=0,1,2,3\dots$ **do**

1. Sample agent trajectories $\tau_i \sim \pi_{\theta_i}$
2. Update the discriminator parameters with the opposite of the gradient:

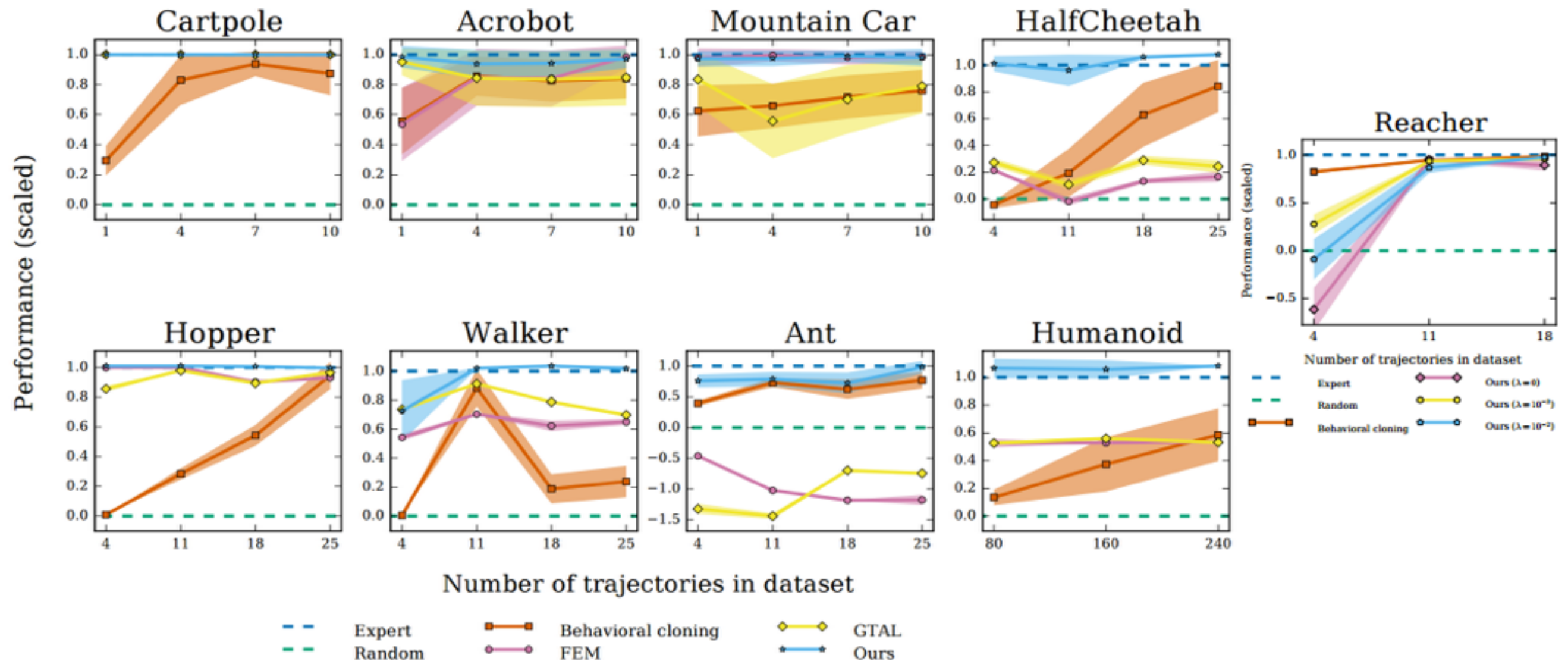
$$\mathbb{E}_{(s,a) \sim \text{Demo}} [\nabla_{\phi} \log(1 - D_{\phi}(s, a))] + \mathbb{E}_{(s,a) \in \tau_i} [\nabla_{\phi} \log(D_{\phi}(s, a))]$$

3. Update the policy using a policy gradient computed with the rewards, e.g., the REINFORCE policy gradient would be:

$$\mathbb{E}_{(s,a) \in \tau_i} [\nabla_{\theta} \log \pi_{\theta} \log D_{\phi_{i+1}}(s, a)]$$

end for

Generative Adversarial Imitation learning



- GAIL: a reinforcement learning method with a reward based on trajectory distribution matching between the agent and an expert.
- BC: reduces imitation learning to supervised learning for individual actions.
- GAIL performs better than behaviour cloning but it requires MORE interactions with the environment.
- Q: Can BC or GAIL outperform the expert?

Generative Adversarial Imitation

learning-action spaces differ between teacher and learner

Find a policy π_θ that makes it impossible for a discriminator network to distinguish between states from the expert demonstrations and states visited by the learnt policy π_θ :

$$\min_{\pi_\theta} \max_D V(D, G) = \mathbb{E}_{(s) \sim \text{Demo}} [\log D(s)] + \mathbb{E}_{(a,s) \sim \pi_\theta} [\log(1 - D(s))]$$

Reward for the policy optimization is how well I matched the demo trajectory distribution, else, how well I confused the discriminator.

$$r(s) = \log D(s)$$

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

Analysis: Optimal discriminator strategy

$$\min_{\color{red}{G}} \max_{\color{blue}{D}} V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log \color{blue}{D}(\color{blue}{x})] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - \color{blue}{D}(\color{red}{G}(\color{red}{z})))]$$

$$V(D, G) = \int_x p_{\text{data}}(x) \log \color{blue}{D}(\color{blue}{x}) dx + \int_z p_z(z) \log(1 - \color{blue}{D}(\color{red}{G}(\color{red}{z}))) dz$$

Optimal discriminator strategy

$$\min_{\textcolor{red}{G}} \max_{\textcolor{blue}{D}} V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log \textcolor{blue}{D}(\textcolor{blue}{x})] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - \textcolor{blue}{D}(\textcolor{red}{G}(\textcolor{red}{z})))]$$

$$V(D, G) = \int_x p_{\text{data}}(x) \log \textcolor{blue}{D}(\textcolor{blue}{x}) dx + \int_z p_z(z) \log(1 - \textcolor{blue}{D}(\textcolor{red}{G}(\textcolor{red}{z}))) dz$$
$$\int_x p_{\text{data}}(x) \log \textcolor{blue}{D}(\textcolor{blue}{x}) dx + \int_x p_G(x) \log(1 - \textcolor{blue}{D}(\textcolor{red}{x})) dx$$

Optimal discriminator strategy

$$\min_{\textcolor{red}{G}} \max_{\textcolor{blue}{D}} V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log \textcolor{blue}{D}(\textcolor{blue}{x})] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - \textcolor{blue}{D}(\textcolor{red}{G}(\textcolor{red}{z})))]$$

$$\begin{aligned} V(D, G) &= \int_x p_{\text{data}}(x) \log \textcolor{blue}{D}(\textcolor{blue}{x}) dx + \int_z p_z(z) \log(1 - \textcolor{blue}{D}(\textcolor{red}{G}(\textcolor{red}{z}))) dz \\ &= \int_x p_{\text{data}}(x) \log \textcolor{blue}{D}(\textcolor{blue}{x}) dx + \int_x p_G(x) \log(1 - \textcolor{blue}{D}(\textcolor{red}{x})) dx \\ &= \int_x p_{\text{data}}(x) \log \textcolor{blue}{D}(\textcolor{blue}{x}) + p_G(x) \log(1 - \textcolor{blue}{D}(\textcolor{red}{x})) dx \end{aligned}$$

Optimal discriminator strategy

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx$$

The discriminator assigns values $D(x)$ to each image x . Let's take the derivative to see where the optimum is attained.

Optimal discriminator strategy

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx$$

$$\frac{d}{dD(x)} \left(p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) \right) = 0$$

Optimal discriminator strategy

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx$$

$$\frac{d}{dD(x)} (p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))) = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} - p_G(x) \frac{1}{1 - D(x)} = 0$$

Optimal discriminator strategy

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx$$

$$\frac{d}{dD(x)} (p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))) = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} - p_G(x) \frac{1}{1 - D(x)} = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} = p_G(x) \frac{1}{1 - D(x)}$$

Optimal discriminator strategy

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx$$

$$\frac{d}{dD(x)} (p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))) = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} - p_G(x) \frac{1}{1 - D(x)} = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} = p_G(x) \frac{1}{1 - D(x)}$$

$$\Leftrightarrow p_{\text{data}}(x)(1 - D(x)) = p_G(x)D(x)$$

Optimal discriminator strategy

$$V(D, G) = \int_x p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x)) dx$$

$$\frac{d}{dD(x)} (p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))) = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} - p_G(x) \frac{1}{1 - D(x)} = 0$$

$$\Leftrightarrow p_{\text{data}}(x) \frac{1}{D(x)} = p_G(x) \frac{1}{1 - D(x)}$$

$$\Leftrightarrow p_{\text{data}}(x)(1 - D(x)) = p_G(x)D(x)$$

$$\Leftrightarrow D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$$

Optimal generator strategy

$$C(G) = \max_D V(G, D)$$

Optimal generator strategy

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \end{aligned}$$

Optimal generator strategy

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\ &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \end{aligned}$$

Optimal generator strategy

$$\begin{aligned}C(G) &= \max_D V(G, D) \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D_G^*(G(z)))] \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)}[\log(1 - D_G^*(x))] \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)}\left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}\right] + \mathbb{E}_{x \sim p_G(x)}\left[\log\left(1 - \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}\right)\right]\end{aligned}$$

Optimal generator strategy

$$\begin{aligned}C(G) &= \max_D V(G, D) \\&= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\&= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \\&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(1 - \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right) \right] \\&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right]\end{aligned}$$

Optimal generator strategy

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(1 - \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] - \log 4 + \log 4 \end{aligned}$$

Optimal generator strategy

$$\begin{aligned}C(G) &= \max_D V(G, D) \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(1 - \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] - \log 4 + \log 4 \\&= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{2p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{2p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] - \log 4\end{aligned}$$

Optimal generator strategy

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(1 - \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] - \log 4 + \log 4 \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{2p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{2p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] - \log 4 \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{\frac{p_{\text{data}}(x) + p_G(x)}{2}} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \frac{p_G(x)}{\frac{p_{\text{data}}(x) + p_G(x)}{2}} \right] - \log 4 \end{aligned}$$

Optimal generator strategy

$$\begin{aligned}
C(G) &= \max_D V(G, D) \\
&= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\
&= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \\
&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(1 - \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right) \right] \\
&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right] \\
&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right] - \log 4 + \log 4 \\
&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{2p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{2p_G(x)}{p_{data}(x) + p_G(x)} \right) \right] - \log 4 \\
&= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \frac{p_G(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] - \log 4 \\
&= D_{KL} \left(p_{data}(x) \parallel \frac{p_{data}(x) + p_G(x)}{2} \right) + D_{KL} \left(p_G(x) \parallel \frac{p_{data}(x) + p_G(x)}{2} \right) - \log 4
\end{aligned}$$

Optimal generator strategy

$$\begin{aligned}
 C(G) &= \max_D V(G, D) \\
 &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D_G^*(G(z)))] \\
 &= \mathbb{E}_{x \sim p_{data}(x)} [\log D_G^*(x)] + \mathbb{E}_{x \sim p_G(x)} [\log(1 - D_G^*(x))] \\
 &= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(1 - \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right) \right] \\
 &= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right] \\
 &= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right] - \log 4 + \log 4 \\
 &= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{2p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{2p_G(x)}{p_{data}(x) + p_G(x)} \right) \right] - \log 4 \\
 &= \mathbb{E}_{x \sim p_{data}(x)} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \frac{p_G(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] - \log 4 \\
 &= D_{KL} \left(p_{data}(x) \parallel \frac{p_{data}(x) + p_G(x)}{2} \right) + D_{KL} \left(p_G(x) \parallel \frac{p_{data}(x) + p_G(x)}{2} \right) - \log 4 \\
 &= 2D_{JSD} (p_{data}(x) \parallel p_G(x)) - \log 4
 \end{aligned}$$

Optimal generator strategy

$$\begin{aligned} C(G) &= \max_D V(G, D) \\ &= \mathbb{E}_{x \sim p_{\text{data}}(x)} \left[\log \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)} \right] + \mathbb{E}_{x \sim p_G(x)} \left[\log \left(\frac{p_G(x)}{p_{\text{data}}(x) + p_G(x)} \right) \right] \\ &= 2D_{\text{JSD}}(p_{\text{data}}(x) || p_G(x)) - \log 4 \end{aligned}$$

Since $D_{\text{JSD}} \geq 0$, $C(G) \geq -\log 4$

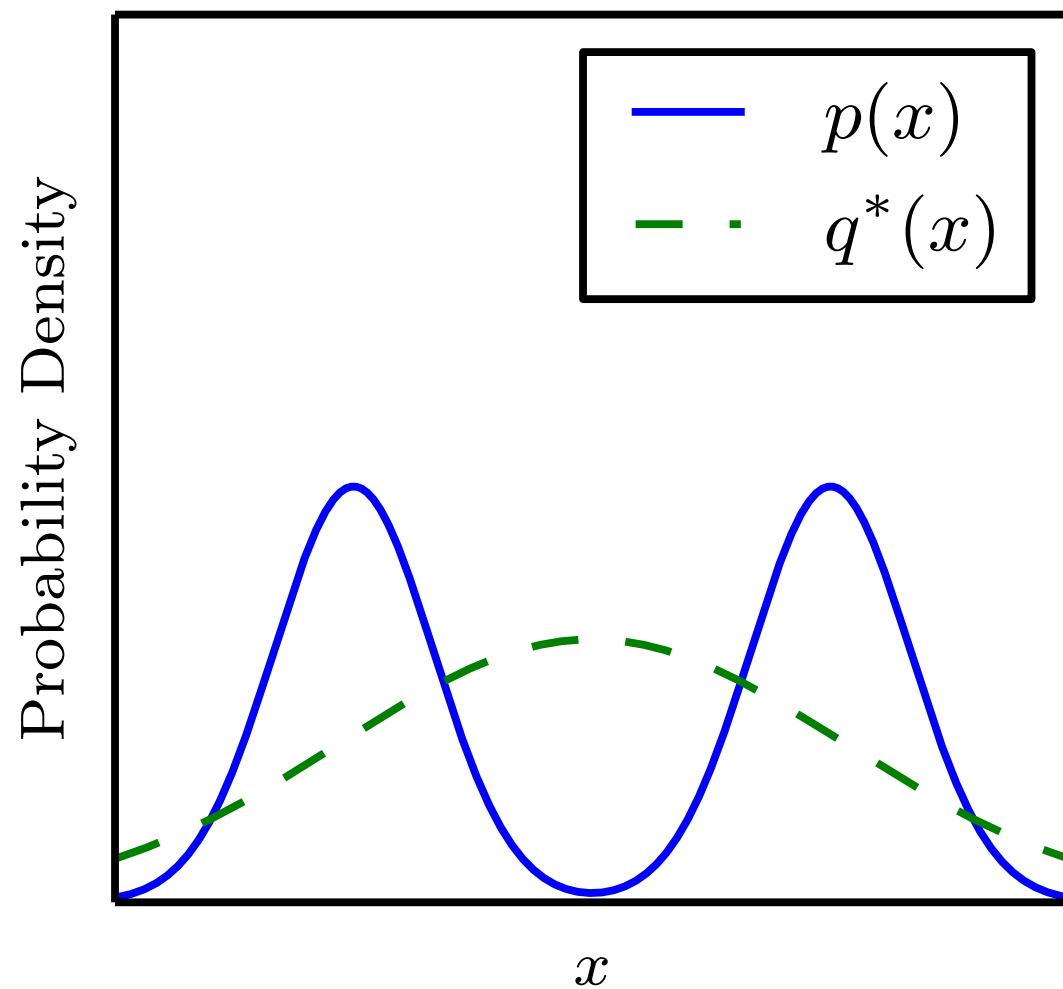
By setting $P_G(x) = p_{\text{data}}(x)$ in the equation above, we get:

$$C(G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} \log \frac{1}{2} + \mathbb{E}_{x \sim p_G(x)} \log \frac{1}{2} = -\log 4$$

Thus generator achieves the optimum when $P_G(x) = p_{\text{data}}(x)$.

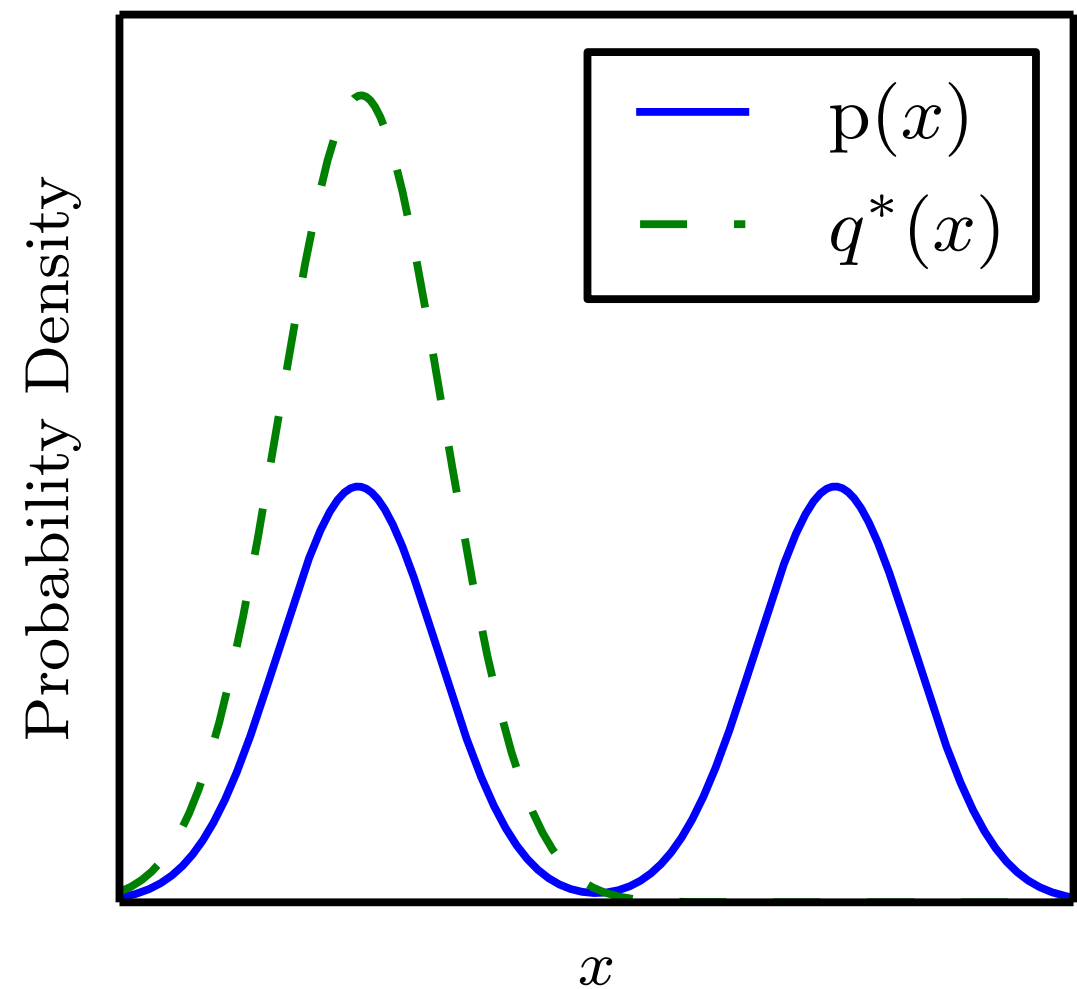
Maybe an explanation of why GANs work

$$q^* = \operatorname{argmin}_q D_{\text{KL}}(p||q)$$



Maximum likelihood

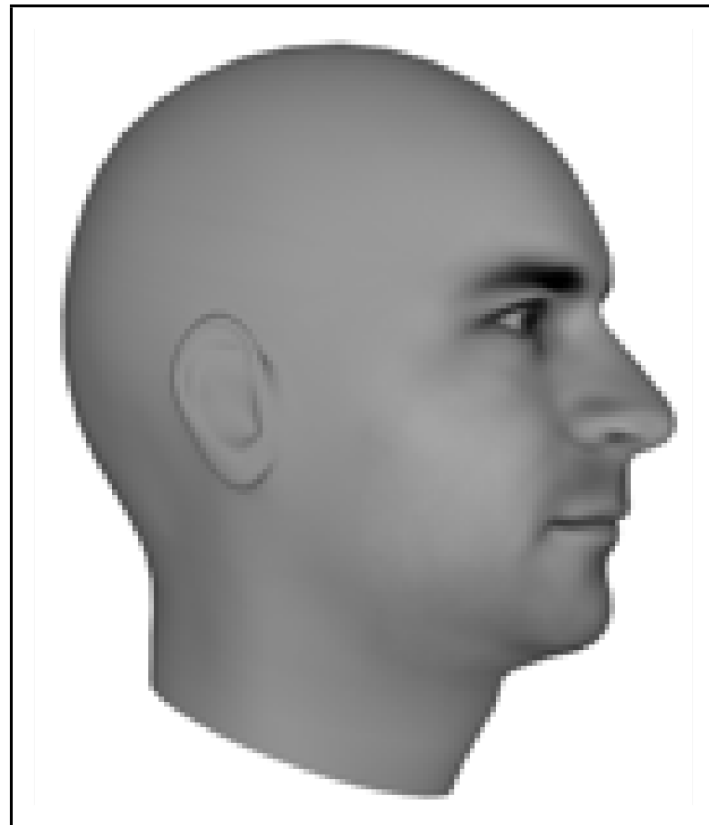
$$q^* = \operatorname{argmin}_q D_{\text{KL}}(q||p)$$



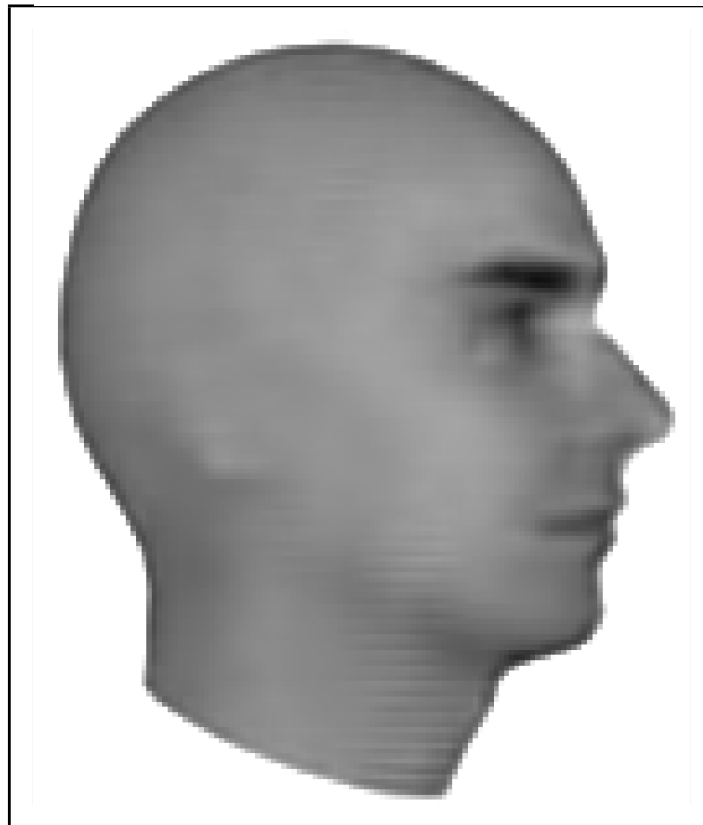
Reverse KL

Next Video Frame Prediction

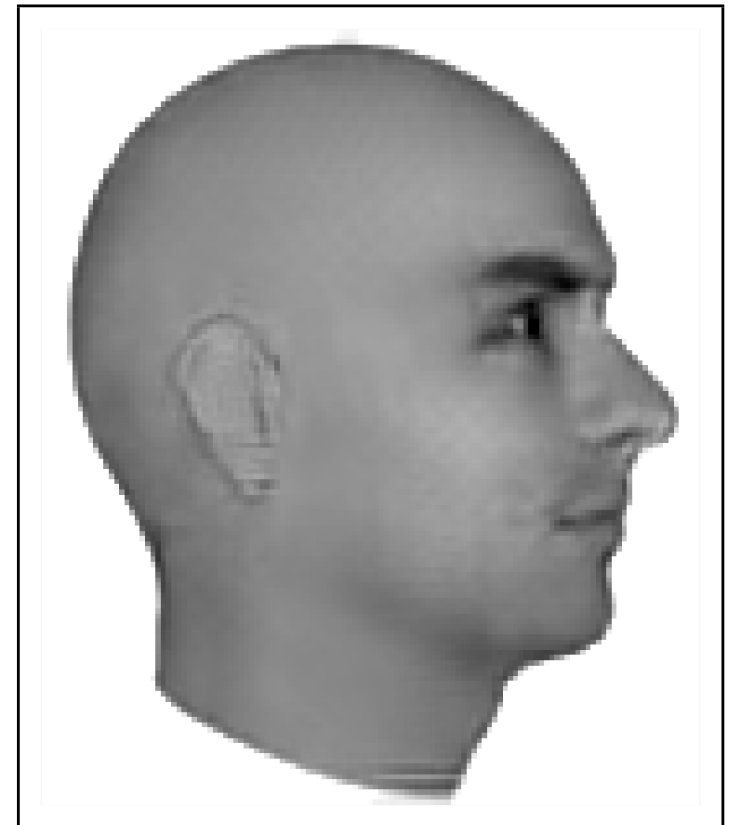
Groundtruth



Max. Likelihood



Adversarial



(Lotter et al 2016)